

Generative Models for Multi-Corruption Image Restoration and Style-Conditioned Face-to-Sketch Synthesis

Eman Ul Huda

Student ID 23i-2559

Department of Computer Science

FAST National University of Computer and Emerging Sciences (FAST-NUCES)

Abstract—This report presents four related generative systems developed, evaluated and deployed as one browser-based application. Tasks 1 to 3 restore 128×128 Oxford-IIIT Pet images affected by salt-and-pepper noise, Gaussian blur or rectangular occlusion: a universal denoising autoencoder (Task 1), a corruption classifier with hard-routed specialist autoencoders (Task 2), and a jointly trained soft mixture of experts (Task 3). On the official test set the universal autoencoder reaches an SSIM of 0.704, and hard routing reaches 0.735 with a classifier accuracy of 99.86%, its gain coming mainly from passing clean images through unchanged. The jointly trained soft mixture of experts raises validation SSIM from 0.783 for hard routing to 0.839 by learning to blend the unchanged input with the experts according to how severe the damage is. Task 4 is a style-conditioned conditional GAN that translates face photographs into sketches in one of three FS2K artist styles, using a U-Net generator, a PatchGAN discriminator and a learned style embedding injected into both networks. On the official FS2K test set it reaches an SSIM of 0.463, an LPIPS of 0.211 and an FID of 37.6, and the true style condition produces the perceptually closest sketch for 72.9% to 95.7% of photographs depending on the style. All models were tuned with Optuna, tracked with Weights & Biases, exported to ONNX and served through a FastAPI backend and a React interface running in Docker containers.

Index Terms—denoising autoencoder, mixture of experts, conditional GAN, image-to-image translation, facial sketch synthesis, Optuna, ONNX

I. INTRODUCTION

Image restoration and image-to-image translation are two common uses of generative models. Restoration recovers a clean image from a corrupted observation; translation maps an image from one visual domain to another while keeping its content. This assignment studies both problems and asks how architectural choices (one shared model, hard routing to specialists, or soft weighting of specialists) and training objectives (pixel, structural, perceptual and adversarial losses) affect quality.

The contributions of this work are: (i) a universal restoration autoencoder trained on runtime-generated corruptions; (ii) a hard-routing system combining a corruption classifier with three specialist autoencoders, evaluated with oracle and predicted routing; (iii) a differentiable soft mixture of experts initialised from the Task 2 components and fine-tuned jointly; (iv) a style-conditioned conditional GAN for face-to-sketch generation; and (v) a containerised web application exposing

all four systems. A central finding of Tasks 1 and 2 is that the bottleneck, not the routing, limits restoration quality, so deciding when not to restore matters as much as restoring well. The soft mixture of experts confirms this: without being told, its gate learned to keep most of a lightly corrupted input and to rely on an expert only as the damage grows.

The source code is available at <https://github.com/eman-huda/genai-assignment1>. The demonstration video is available at <https://youtu.be/B2mOOliPXoI>.

II. RELATED WORK

Denoising autoencoders and restoration. Denoising autoencoders learn robust representations by reconstructing clean inputs from corrupted ones [1]. Convolutional encoder-decoders with skip connections are widely used for restoration [2], and residual CNNs such as DnCNN handle blind Gaussian denoising [3]. Context encoders address large missing regions [4]. All-in-one restoration models handle several unknown degradations with a single network [5], which is the setting of Task 1.

Mixtures of experts. Mixtures of experts combine specialised sub-networks through a gating network [6]. Sparse gating scales this idea to very large models [7], and auxiliary load-balancing losses prevent the gate from collapsing onto a few experts [7], [8]. Task 3 uses a dense softmax gate with a balance regulariser of this kind.

Conditional GANs and sketch synthesis. GANs train a generator against a discriminator [9]; conditional GANs feed side information to both [10]. Pix2pix combines a U-Net generator [11], a PatchGAN discriminator and an L1 term for paired translation [12]. Early face sketch synthesis relied on patch-based exemplar methods [13]. FS2K is the largest public facial sketch dataset with artist-drawn sketches in three styles; its authors' FSGAN conditions on style by expanding a style vector into a spatial map [14], the approach adopted in Task 4. Projection discriminators are an alternative way to condition the discriminator on a class [15].

Evaluation. SSIM measures structural similarity [16], LPIPS measures perceptual distance with deep features [17], and FID compares distributions of Inception features [18]. FID is biased for small sample sizes [19], which motivates the Kernel Inception Distance (KID) [20].

TABLE I
CORRUPTION CONFIGURATION (TASKS 1 TO 3).

Condition	Training (sampled per load)	Test levels (low / med / high)
Clean	No corruption	n/a
Salt-and-pepper	$p \sim U(0.02, 0.15)$, black or white with equal probability	$p = 0.03/0.08/0.15$
Gaussian blur	$k \in \{3, 5, 7\}$, $\sigma \sim U(0.5, 2.5)$	$(3, 0.7)/(5, 1.5)/(7, 2.5)$
Occlusion	1 to 3 black rectangles, union covers 10% to 35%	1 / 2 / 3 rectangles, $\approx 10\%$ / 20% / 35%

Hyperparameter optimisation. Optuna provides define-by-run search with pruning [21]; its default sampler is the Tree-structured Parzen Estimator [22], and parameter importances are estimated with fANOVA [23].

III. DATASETS AND PREPROCESSING

A. Oxford-IIIT Pet with runtime corruptions (Tasks 1 to 3)

The Oxford-IIIT Pet dataset [24] contains images of 37 cat and dog breeds; class labels are not used. The official trainval set was split 80/20 into training and validation sets with seed 42 (2,944 training and 736 validation images), and the official test set (3,669 images) was used only for final evaluation. All images were converted to RGB and resized to 128×128 . The same split is used for Tasks 1, 2 and 3.

During training, each loaded image receives one of four conditions with equal probability, sampled again every time the image is loaded (Table I). Validation corruptions are fixed in a manifest that stores the type, severity, random seed, blur settings and rectangle coordinates of every image. The test manifest contains each test image ten times: once clean and three fixed severities for each corruption. Rectangles are placed until their union reaches the target coverage within a small tolerance; across the test manifest the achieved occlusion coverage averaged 10.0% (range 9.8% to 10.3%), 19.8% (18.5% to 20.5%) and 34.5% (33.5% to 35.7%) for the three levels. The validation manifest holds 184 images of each condition. Fig. 1 shows one image under every test level.

B. FS2K (Task 4)

FS2K contains 2,104 photograph-sketch pairs drawn by professional artists in three styles [14]: 1,058 pairs in the official training set and 1,046 in the official test set. The official definitions were kept. A validation set of 15% of the official training portion was drawn with seed 42, stratified by style (Table II). The official test set was used only once, for the final evaluation.

Each annotation entry was matched to its files using the naming rule of the authors’ split script (photo photo1/image0110 corresponds to sketch sketch1/sketch0110). All 2,104 pairs were found, with no missing files and no size mismatch between a photograph and its sketch. Images were converted to RGB (greyscale images were expanded and transparent images composited

TABLE II
FS2K SPLITS USED IN TASK 4 (PAIRS PER STYLE).

Split	Style 1	Style 2	Style 3	Total
Training (85% of official train)	303	298	298	899
Validation (15%, stratified)	54	52	53	159
Test (official)	619	381	46	1,046

on white), resized to 128×128 with area interpolation, and scaled to $[-1, 1]$.

Training augmentation follows pix2pix [12]: both images are resized to 142×142 , cropped back to 128×128 at the same random offset, and flipped horizontally together with probability 0.5. Applying one sampled transformation to both images preserves the pixel correspondence that paired training relies on; Fig. 3 confirms visually that the augmented photograph and sketch remain aligned.

Fig. 2 shows that the three styles differ mainly in stroke weight and shading. Style 1 uses light, thin outlines with little shading; Style 2 uses dark, heavy strokes and dense hatching, especially in the hair; Style 3 lies between the two, with moderate shading and more facial detail. The official test set is strongly imbalanced, with only 46 Style 3 pairs, which affects how reliable per-style test statistics are (Section VII-F).

IV. TASK 1: UNIVERSAL MULTI-CORRUPTION DENOISING AUTOENCODER

A. Architecture

The network is a convolutional autoencoder with a genuine bottleneck (Fig. 4). A stem of two 3×3 convolutions (BatchNorm, ReLU) keeps the full 128×128 resolution with c channels. Four down-blocks, each a stride-2 convolution followed by a 3×3 convolution, reduce the resolution to 64, 32, 16 and 8 while the channels grow to $2c, 4c, 8c, 8c$. A 1×1 convolution then compresses the 8×8 features to L latent channels. The decoder mirrors the encoder: a 1×1 convolution back to $8c$ channels and four up-blocks of bilinear upsampling followed by two convolutions, which avoids the checkerboard artefacts of transposed convolutions [29]. A 3×3 convolution with a sigmoid outputs the RGB image. Spatial dropout follows every block.

Optuna selected $c = 48$ and $L = 64$, so the latent tensor has $64 \times 8 \times 8 = 4,096$ values: 8.3% of the $3 \times 128 \times 128 = 49,152$ input values, a 12:1 compression. The model has 9,308,515 parameters. No skip connection bypasses the bottleneck, so every output pixel must be reconstructed from the latent code.

B. Loss function

With clean image x , corrupted input $\tilde{x}$ and reconstruction $\hat{x} = D(E(\tilde{x}))$, the training loss is

$$\mathcal{L}_{\text{rec}} = \alpha \mathcal{L}_1(x, \hat{x}) + (1 - \alpha) (1 - \text{SSIM}(x, \hat{x})). \quad (1)$$

The L1 term enforces pixel accuracy and the SSIM term [16] (11×11 Gaussian window, $\sigma = 1.5$) preserves edges, local contrast and structure. The model is never told which corruption was applied.

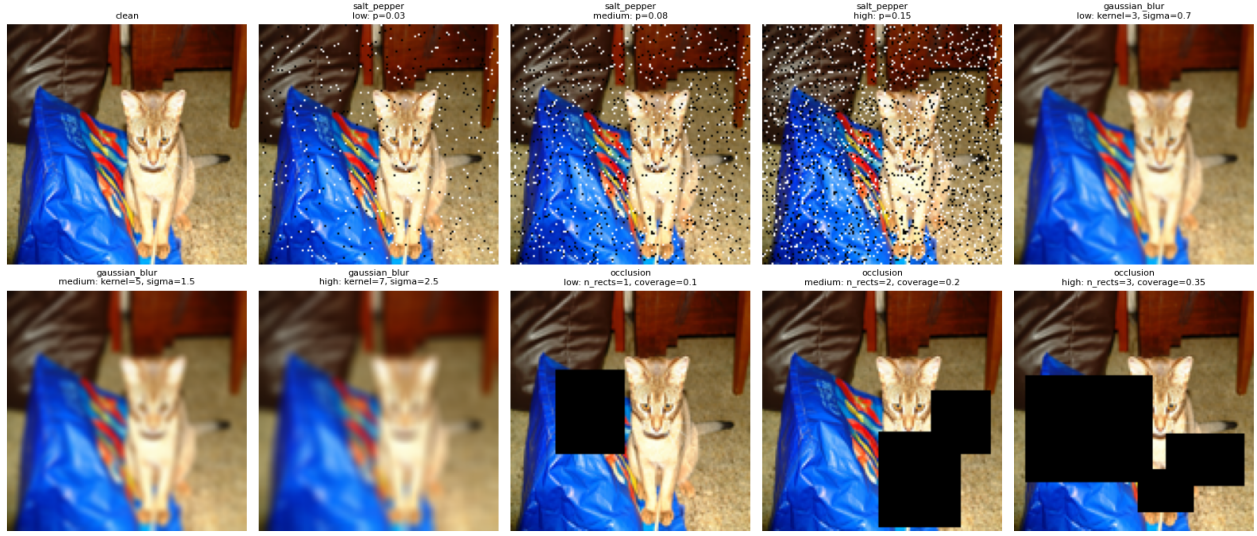


Fig. 1. One validation image under every fixed test level: clean, salt-and-pepper ($p = 0.03, 0.08, 0.15$), Gaussian blur (kernel and $\sigma = (3, 0.7), (5, 1.5), (7, 2.5)$) and occlusion (1, 2, 3 rectangles covering about 10%, 20%, 35%).

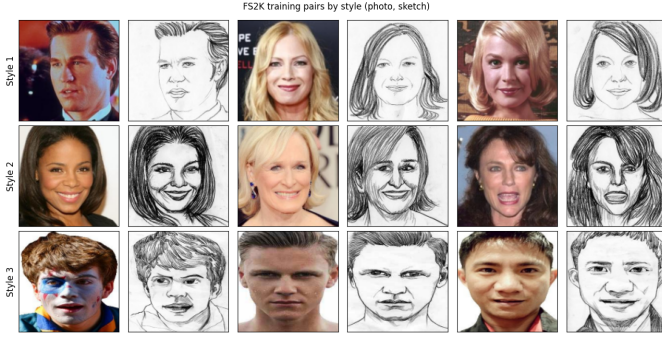


Fig. 2. FS2K training pairs, three per style (photograph left, sketch right).

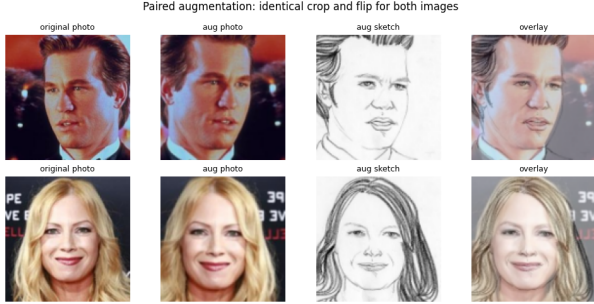


Fig. 3. Paired augmentation check: the overlay of the augmented photograph and sketch shows that both received the same crop and flip.

C. Training and hyperparameter optimisation

Training uses Adam with the runtime corruption pipeline of Section III: every loaded image receives a freshly sampled condition and severity. Optuna [21] searched six hyperparameters (Table III) with the TPE sampler [22] and median pruning, using 25 trials of 8 epochs each. Trials were compared on the fixed validation objective $L1 + (1 - \text{SSIM})$, which does not

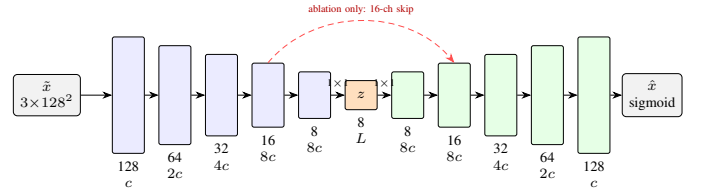


Fig. 4. Task 1 autoencoder. Numbers give spatial size and channels ($c = 48, L = 64$). Strided-convolution encoder (blue), 1×1 bottleneck (orange), upsampling decoder (green). The dashed skip was used only in the ablation of Section IV-D.

TABLE III
TASK 1 OPTUNA SEARCH SPACE AND SELECTED VALUES (BEST OF 25 TRIALS, 8 EPOCHS EACH).

Hyperparameter	Search range	Selected
Learning rate	log-uniform $[10^{-4}, 3 \times 10^{-3}]$	4.21×10^{-4}
Batch size	$\{16, 32, 64\}$	16
Latent channels L	$\{8, 16, 32, 64\}$	64
Base channels c	$\{16, 32, 48\}$	48
Dropout	uniform $[0, 0.3]$	0.219
Loss weight α	uniform $[0.5, 0.95]$	0.508
Validation objective		0.4561 (trial 18)

depend on the tuned α ; otherwise a trial could improve its score by reweighting the metric rather than by restoring better.

Of the 25 trials, 23 completed and 2 were pruned; the objective fell from 0.608 (trial 0) to 0.456 (trial 18) (Fig. 5). Every competitive trial used $c = 48$ and batch size 16, and the two completed trials with the smallest latent ($L = 8$) scored 0.480 and 0.487, behind the best $L = 64$ trials (0.456 to 0.467). This shows that latent capacity limits quality. The fANOVA importances [23] attribute 0.65 to dropout and 0.30 to α ; the categorical parameters receive almost none because TPE quickly concentrated on $c = 48$ and batch size 16, so

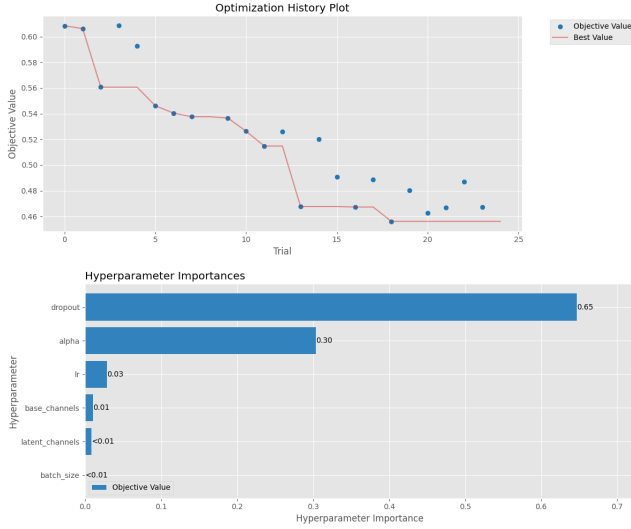


Fig. 5. Task 1 Optuna study: objective per trial (top) and fANOVA importance over completed trials (bottom).

TABLE IV
SKIP-CONNECTION ABLATION (VALIDATION, 20 EPOCHS EACH).

Skip	PSNR	SSIM	SSIM by condition			
			Clean	S&P	Blur	Occl.
None	20.86	0.658	0.674	0.681	0.684	0.594
16 channels	22.17	0.771	0.801	0.799	0.796	0.686

their effect was not resolved rather than shown to be absent. Optuna also pushed α to the low end of its range (0.508), giving the SSIM term almost half the weight. The starting value of 0.8 suggested in the brief would have favoured pixel accuracy over structure.

D. Skip-connection ablation

The brief allows limited skip connections only if their effect is investigated. The selected configuration was trained twice for 20 epochs: without a skip, and with one narrow skip that projects the 16×16 encoder features to 16 channels and joins them to the decoder (dashed in Fig. 4). Table IV shows that the skip improved every condition, including salt-and-pepper noise and occlusion. A skip at full resolution would pass noise straight through, but this one starts after three strided blocks, where the features are already filtered, so it did not leak corruption into the output.

Despite this gain of 0.11 in validation SSIM, the final model uses no skip. The 16-channel skip at 16×16 carries $16 \times 16 \times 16 = 4,096$ values, exactly as many as the latent code itself. It would therefore double the information that reaches the decoder and let a large part of the reconstruction bypass the bottleneck, weakening the autoencoder requirement of the brief. Keeping a strict bottleneck also makes Tasks 1, 2 and 3 directly comparable, because the Task 2 specialists use the same skip-free architecture. The ablation is therefore reported

TABLE V
TASK 1 TEST RESULTS BY CONDITION. “WORSE” IS THE SHARE OF TEST ENTRIES WHERE THE OUTPUT SSIM IS BELOW THE INPUT SSIM.

Condition	In PSNR	Out PSNR	In SSIM	Out SSIM	Worse
Clean	∞	21.75	1.000	0.735	100%
Salt-and-pepper	16.38	21.79	0.382	0.731	7.1%
Gaussian blur	27.82	21.80	0.814	0.717	80.0%
Occlusion	13.64	19.63	0.713	0.654	66.6%
All		21.14		0.704	

TABLE VI
TASK 1 TEST SSIM BY CORRUPTION AND SEVERITY.

Corruption	Severity	In SSIM	Out SSIM	Out PSNR
Salt-and-pepper	low ($p=0.03$)	0.603	0.735	21.79
	medium ($p=0.08$)	0.340	0.732	21.81
	high ($p=0.15$)	0.204	0.727	21.77
Gaussian blur	low (3, 0.7)	0.944	0.735	21.86
	medium (5, 1.5)	0.806	0.725	21.91
	high (7, 2.5)	0.692	0.690	21.63
Occlusion	low (10%)	0.862	0.698	20.67
	medium (20%)	0.731	0.661	19.72
	high (35%)	0.546	0.602	18.51

as evidence of how much the bottleneck costs, rather than adopted.

E. Final training

The selected configuration was trained for 60 epochs with early stopping (patience 12) on the validation objective. The best epoch was the last (validation objective 0.3513), and the curves in Fig. 6 were still improving slowly, so a longer schedule would probably have helped a little. Validation SSIM lies slightly above training SSIM throughout, because dropout is active during training and the training corruptions are freshly resampled every epoch. Per-condition validation PSNR shows occlusion about 2.5 dB below the other conditions for the whole run, which is the gap that also appears on the test set.

F. Results

Table V reports the official test set (36,690 entries: 3,669 images under 10 conditions). Overall the model reaches PSNR 21.14 dB, SSIM 0.704 and L1 0.066. Table VI separates the severity levels.

The most important observation is that output quality is almost independent of the input. Salt-and-pepper outputs reach SSIM 0.735, 0.732 and 0.727 whether the input SSIM was 0.60 or 0.20, and clean, noisy and blurred inputs all end near SSIM 0.73 and PSNR 21.8 dB. The output is therefore limited by the 4,096-value bottleneck, not by the corruption. The consequences differ by corruption. Salt-and-pepper noise is removed almost completely (SSIM +0.35, worse in only 7.1% of cases), because isolated extreme pixels carry little information and do not survive compression. Mild corruptions are made worse: a clean image loses 0.26 SSIM, low blur loses 0.21, and 80% of blurred inputs leave the model with lower



Fig. 6. Task 1 final training: loss, training and validation SSIM, and validation PSNR per condition.

SSIM than they entered. Occlusion is the hardest condition (SSIM 0.654), because the model must invent content for up to 35% of the image. Only at the highest occlusion level does the output beat the input SSIM.

Fig. 7 shows twelve representative test entries (closest to the median SSIM of each condition and severity). Two systematic effects are visible beyond the scores. Fine texture such as fur is smoothed away, and colours are strongly desaturated: the red collar in the fourth row loses most of its colour. Both are consequences of compression, since the code keeps coarse shape and luminance better than texture and chroma. The absolute error maps confirm this, with errors concentrated on edges, texture and saturated colours.

Failure cases. Fig. 8 shows the lowest-SSIM entry for each condition. Three of the four failures are the same photograph: a dog in a meadow of flowers and grass. It fails as a clean image (SSIM 0.366), with salt-and-pepper noise (0.356) and with blur (0.296), so the failure is caused by image content rather than by the corruption. Dense high-frequency texture cannot be encoded in the latent code, and the decoder produces a smooth, colourless pattern. The fourth failure is a cat on a black background with heavy occlusion (SSIM 0.243). The model fills the occluded region and much of the dark background with light grey, the brightness of an average training image, so almost the whole background becomes error. Both failures show that a single compressed code learns a generic image prior that suits typical pet photographs but not unusual texture or lighting.

G. ONNX export

The final model was exported to ONNX (opset 17, dynamic batch size; input `input`, output `output`, RGB in $[0, 1]$). On 80 test entries covering every condition and severity, the maximum absolute difference to PyTorch was 7.2×10^{-7} . Single-image CPU latency on Colab was 110.8 ms with ONNX Runtime against 140.3 ms in PyTorch.

V. TASK 2: CORRUPTION CLASSIFICATION AND HARD-ROUTED SPECIALISTS

A. System overview

Fig. 9 shows the hard-routing system. A convolutional classifier predicts one of four classes; an argmax selects

one branch. A clean prediction returns the input unchanged (identity bypass, no expert is run), and each corruption class sends the image to its own specialist autoencoder.

B. Corruption classifier

The classifier has four stages of two 3×3 convolutions (BatchNorm, ReLU), with max pooling between stages. The first stage runs at full resolution before any pooling, so isolated noise pixels and the edge sharpness that blur removes are still visible to it. Global average and global max pooling are concatenated, because a black occlusion box or a single noise pixel is a local, extreme response that average pooling alone would dilute; dropout and a linear layer then give four logits. Training labels come from the runtime corruption pipeline. A balanced batch sampler gives every batch exactly one quarter of each class while the severity is still sampled freshly (Fig. 10). The loss is multiclass cross-entropy, optimised with AdamW, cosine learning-rate decay and mixed precision; model selection uses validation macro-F1.

Optuna searched the learning rate, batch size, channel configuration, dropout and weight decay (Table VII) with 25 trials of 8 epochs; 11 completed and 14 were pruned. The task turned out to be easy: every completed trial reached a validation macro-F1 of at least 0.9946, and the best (trial 22) reached 1.000 on the 736 validation images. With differences of only one to four validation images between trials, the search could not separate configurations reliably, and the fANOVA importance (0.78 for dropout) should not be over-interpreted. The selected network is the smallest channel configuration (16, 32, 64, 128) with 295,028 parameters. The final classifier was trained for up to 40 epochs with patience 8; it stopped after epoch 25, and its best epoch was 17 (validation accuracy and macro-F1 0.9986, Fig. 11). The validation spike at epoch 5 is a transient instability early in training that recovered by the next epoch.

Classifier results. On the test set the classifier reaches an accuracy of 0.9986, macro-precision 0.9984, macro-recall 0.9970 and macro-F1 0.9977 (Table VIII, Fig. 12). Salt-and-pepper noise is recognised perfectly. All of the remaining errors involve the clean class: 1.1% of clean images are taken for occlusion (0.7%) or blur (0.4%), and a few low-severity blurred or occluded images are taken for clean. Fig. 13 shows

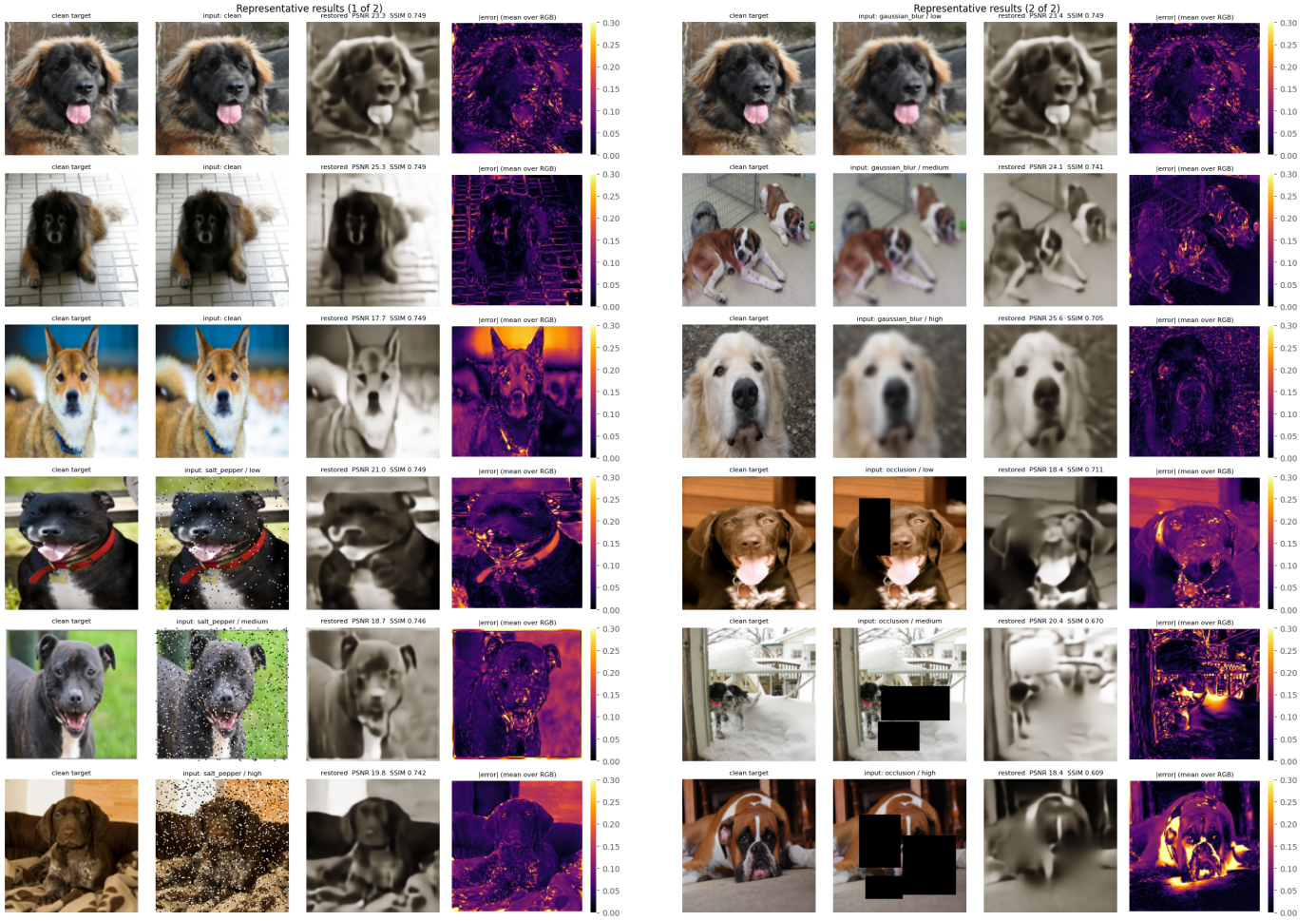


Fig. 7. Task 1: twelve representative test results (clean target, model input, restoration, absolute error map). Left: clean and salt-and-pepper; right: Gaussian blur and occlusion, at each severity.

TABLE VII
TASK 2 CLASSIFIER: OPTUNA SEARCH SPACE AND SELECTED VALUES.

Hyperparameter	Search range	Selected
Learning rate	log-uniform $[10^{-4}, 3 \times 10^{-3}]$	2.94×10^{-4}
Batch size	{32, 64, 128}	32
Channels	4 configurations ^a	small
Dropout	uniform $[0, 0.5]$	0.113
Weight decay	log-uniform $[10^{-6}, 10^{-2}]$	1.17×10^{-5}

^asmall (16, 32, 64, 128), medium (32, 64, 128, 256), wide (48, 96, 192, 256), deep-narrow (32, 64, 96, 128).

TABLE VIII
TASK 2 CLASSIFIER TEST RESULTS PER CLASS.

Class	Precision	Recall	F1	Support
Clean	0.9973	0.9888	0.9930	3,669
Salt-and-pepper	1.0000	1.0000	1.0000	11,007
Gaussian blur	0.9986	0.9995	0.9991	11,007
Occlusion	0.9976	0.9995	0.9985	11,007
Macro average	0.9984	0.9970	0.9977	36,690

the most confident mistakes. Clean photographs with large black regions (dark backgrounds, black cats, letterbox borders) are called occlusion, because they contain exactly the uniform black areas the occlusion class is defined by. Naturally soft or out-of-focus photographs are called blur. A blur of kernel 3 and $\sigma = 0.7$ is barely visible and was called clean. These errors are ambiguities in the data rather than arbitrary mistakes.

C. Specialist autoencoders

The three specialists use the Task 1 architecture with independently trained parameters, each trained only on its own corruption with the clean image as target and the loss of (1). To keep the search feasible, one shared Optuna study selected a common configuration, scored by the mean validation objective of all three specialists (15 trials of 8 epochs; 10 completed, 5 pruned). Dropout and the absence of skips were inherited from Task 1 so that the specialists differ from the universal model only in what they are trained on. The study selected

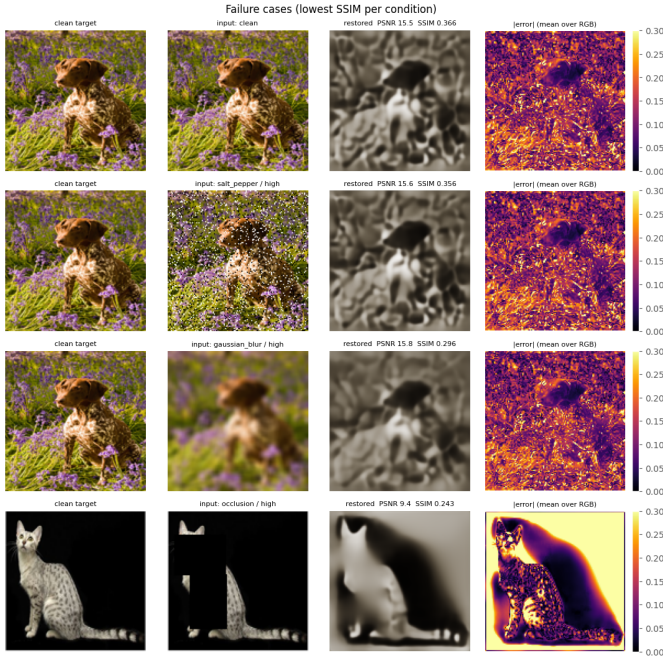


Fig. 8. Task 1 failure cases: lowest test SSIM per condition.

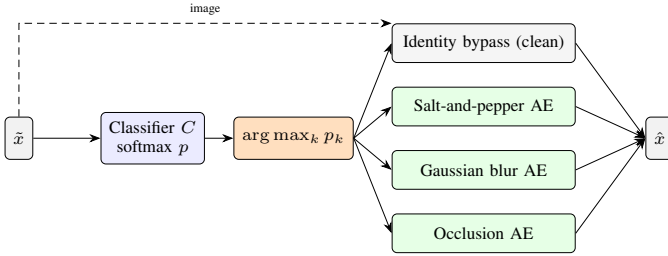


Fig. 9. Task 2 hard routing: the classifier’s most probable class selects exactly one branch. Only the selected specialist is run.

learning rate 3.13×10^{-4} , batch size 16, $c = 48$, $L = 16$ and $\alpha = 0.526$ (objective 0.4550, trial 14; search ranges as in Table III). The fANOVA importance was 0.76 for α and 0.23 for the learning rate (Fig. 14).

The specialists use a four-times smaller latent code than the universal model (1,024 values, 48:1 compression; 9,271,603 parameters each). A specialist only has to undo one kind of damage, so it needs less capacity for the same quality. Each specialist was trained for 60 epochs with patience 12 (Fig. 15). The salt-and-pepper and blur specialists both reached validation SSIM 0.744, the occlusion specialist 0.643 (Table IX), and all three were still improving slightly at the end of training.

D. Hard-routed restoration: oracle and predicted routing

In oracle mode the true label from the test manifest selects the branch; in predicted mode the classifier selects it. Table X compares both, and Table XI compares them with the Task 1 universal model on exactly the same test entries.

Because the classifier is almost perfect, predicted routing is practically identical to oracle routing: across all corrupted



Fig. 10. Eight images from one balanced classifier batch: each of the four classes appears equally often, with freshly sampled severities.

TABLE IX
SPECIALISTS ON THEIR OWN VALIDATION CORRUPTION.

Specialist	In PSNR	Out PSNR	In SSIM	Out SSIM
Salt-and-pepper	16.13	22.20	0.359	0.744
Gaussian blur	28.63	22.39	0.844	0.744
Occlusion	13.17	19.63	0.697	0.643

entries the SSIM lost to routing errors is below 0.001. Almost all of the gain over Task 1 comes from the identity bypass. Clean images keep SSIM 0.997 instead of 0.735, which lifts the overall test SSIM from 0.704 to 0.735. For corrupted images the specialists are only slightly better than the universal model (+0.002 to +0.010 SSIM), with the largest gain for high blur (0.714 against 0.690). Both systems share the same limitation: the specialists also produce outputs near SSIM 0.73 whatever the input, and for low blur they still degrade the image (0.944 input against 0.735 output). Specialisation therefore lets a smaller code do the same job, but it does not raise the quality ceiling set by the bottleneck. Fig. 16 shows representative hard-routed results. The identity bypass returns clean images untouched, and the specialist outputs show the same smoothing and desaturation as Task 1.

Failures caused by classifier errors. Only 52 of the 36,690 test entries (0.14%) were misrouted (Table XII), and none of them was harmless: every routing error changed SSIM by at least 0.01. The costly errors are clean images sent to a specialist. A clean image sent to the occlusion or blur expert loses on average 0.27 or 0.22 SSIM, because a specialist always reconstructs through its bottleneck (Fig. 17). The worst cases are again photographs with black borders or dark backgrounds, and soft, low-contrast scenes. The reverse error has the opposite effect. When a low-severity blurred or occluded image was taken for clean, the identity bypass returned the mildly damaged input, which scored 0.28 and 0.12 SSIM *higher* than the correct specialist would have achieved. This confirms the observation of Task 1: for light corruption, not restoring is better than restoring through the bottleneck, and a system that can weigh the identity branch against the experts could exploit this, which motivates the soft routing of Task 3.

E. ONNX export

The classifier (wrapped to output both logits and probabilities) and the three specialists were exported to ONNX (opset 17, dynamic batch size). On 80 test entries the classifier’s probabilities differed from PyTorch by at most 1.5×10^{-7} with identical predicted classes, and the specialists by at most 0.9×10^{-6} to 2.1×10^{-6} . On the Colab CPU the classifier

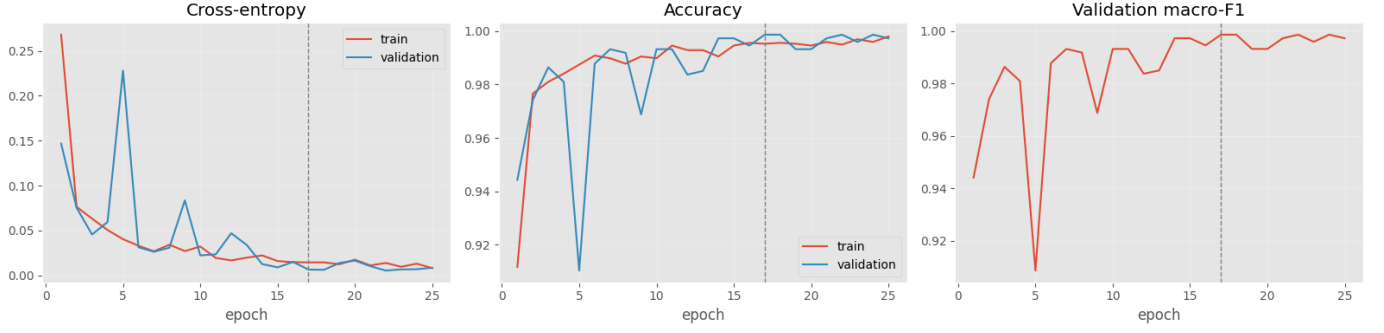


Fig. 11. Task 2 classifier training: cross-entropy, accuracy and validation macro-F1. The dashed line marks the selected epoch.

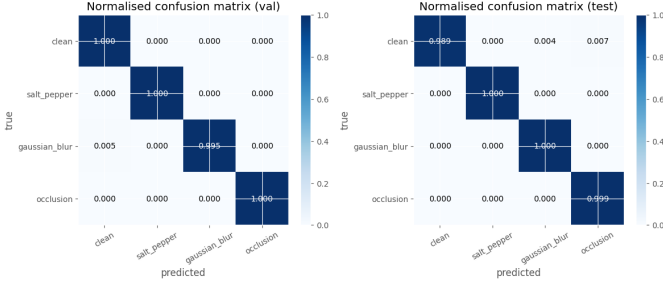


Fig. 12. Row-normalised confusion matrices of the classifier on the validation set (left) and the test set (right).

TABLE X

TASK 2 ON THE TEST SET: ROUTING ACCURACY AND SSIM WITH ORACLE AND PREDICTED ROUTING.

Condition	Routing acc.	In SSIM	Oracle	Predicted	Pred. PSNR
Clean	0.9888	1.000	1.000	0.997	n/a
Salt-and-pepper	1.0000	0.382	0.736	0.736	21.85
Gaussian blur	0.9995	0.814	0.727	0.727	21.81
Occlusion	0.9995	0.713	0.656	0.656	19.77
All corrupted	0.9997	0.636	0.706	0.706	21.14

takes 5.9 ms per image and a specialist 110 to 134 ms. The classifier adds only about 5% to the cost of a restoration, and the identity bypass skips the expert entirely for clean images.

VI. TASK 3: JOINTLY TRAINED SOFT MIXTURE OF EXPERTS

A. Model

The gating network g is initialised from the Task 2 classifier (295,028 parameters) and outputs four logits. Routing weights are

$$w = \text{softmax}(g(\tilde{x})/T), \quad w = [w_0, w_1, w_2, w_3], \quad (2)$$

and the restored image is

$$\hat{x} = w_0 \tilde{x} + w_1 E_{\text{salt}}(\tilde{x}) + w_2 E_{\text{blur}}(\tilde{x}) + w_3 E_{\text{occ}}(\tilde{x}), \quad (3)$$

where branch 0 is an identity branch and the three experts (27,814,809 parameters in total) are initialised from the Task 2 specialists. Unlike hard routing, every branch is evaluated for

TABLE XI

TEST SSIM OF THE TASK 1 UNIVERSAL MODEL AND TASK 2 HARD ROUTING (PREDICTED) ON THE SAME ENTRIES.

Condition	Task 1 universal	Task 2 predicted	Difference
Clean	0.735	0.997	+0.262
Salt-and-pepper	0.731	0.736	+0.005
Gaussian blur	0.717	0.727	+0.010
Occlusion	0.654	0.656	+0.002
All 36,690 entries	0.704	0.735	+0.031

TABLE XII

MISROUTED TEST ENTRIES BY TRUE AND PREDICTED CLASS. SSIM LOST IS ORACLE MINUS PREDICTED (NEGATIVE VALUES MEAN THE ROUTING ERROR HELPED).

True	Predicted	n	Mean confidence	SSIM lost
Clean	Gaussian blur	14	0.764	0.221
Clean	Occlusion	27	0.753	0.265
Gaussian blur	Clean	5	0.748	-0.284
Occlusion	Clean	5	0.733	-0.115
Occlusion	Gaussian blur	1	0.950	0.106

every image, and because the combination is differentiable the gate and experts can be trained jointly through the reconstruction error. Before training, the loaded components reproduced the Task 2 results on the validation set (classifier accuracy 0.9986; hard-routing SSIM 0.7831 predicted and 0.7826 oracle), which confirmed that they were loaded correctly.

B. Loss

$$\mathcal{L}_{\text{MoE}} = \lambda_1 \mathcal{L}_1 + \lambda_2 (1 - \text{SSIM}) + \lambda_3 \mathcal{L}_{\text{CE}} + \lambda_4 \mathcal{L}_{\text{balance}}, \quad (4)$$

with $\mathcal{L}_{\text{balance}} = \sum_{k=0}^3 (\bar{w}_k - \frac{1}{4})^2$, where $\bar{w}_k$ is the mean weight of branch k in a batch. A balanced batch sampler gives every batch exactly one quarter of each condition, so perfect routing gives $\bar{w}_k = 1/4$ and the balance term is zero at that solution; it only penalises collapse onto a subset of branches, in the spirit of load-balancing losses [7], [8]. The cross-entropy term is applied to the raw gate logits, so it ties the gate to the corruption label while T alone controls the sharpness of the routing. The reconstruction weighting is $\lambda_1 = a$, $\lambda_2 = 1 - a$.

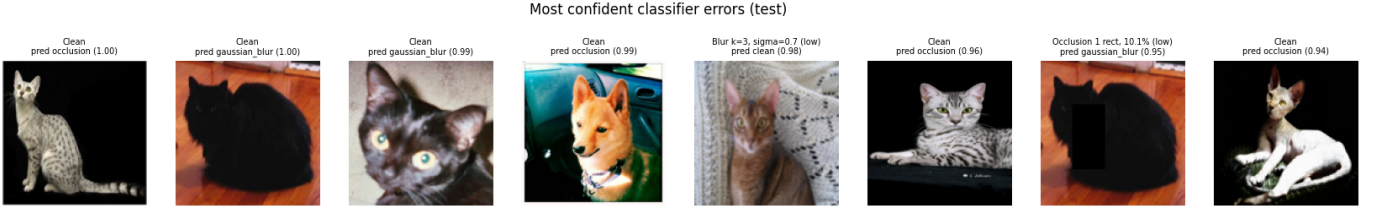


Fig. 13. The most confident classifier errors on the test set (true class, predicted class and confidence).

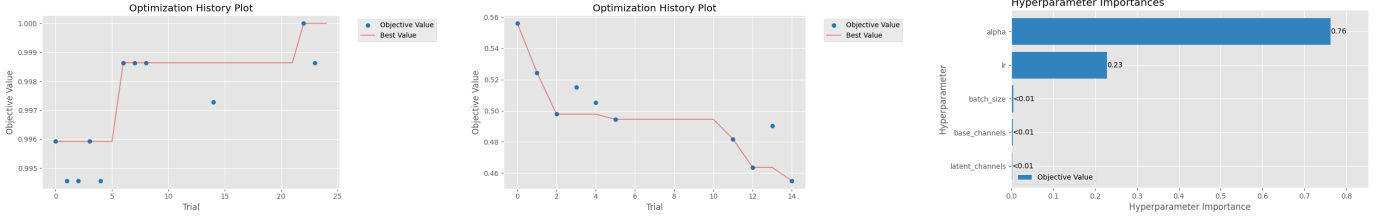


Fig. 14. Task 2 Optuna studies: classifier history (left, validation macro-F1, maximised), specialist history (centre, mean validation objective, minimised) and specialist hyperparameter importance (right).

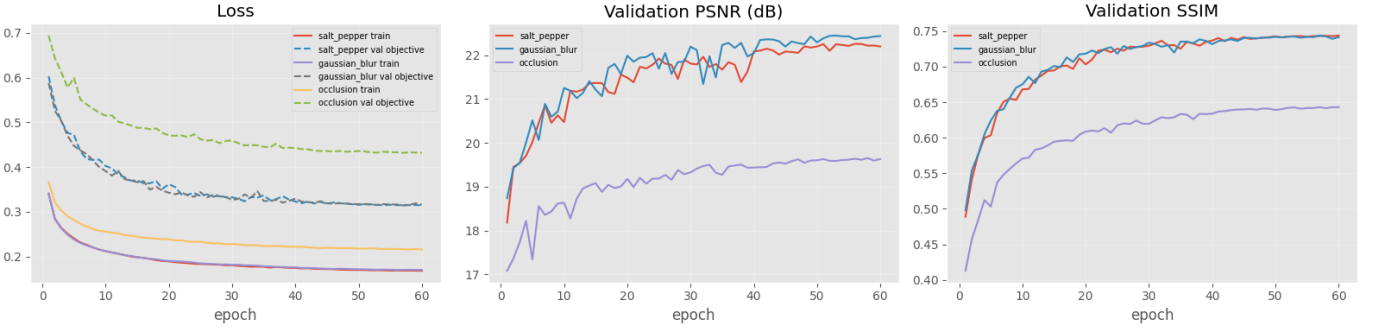


Fig. 15. Training of the three specialists: training loss and validation objective, PSNR and SSIM.

C. Training and hyperparameter optimisation

Training starts with a warm-up in which the experts are frozen (no gradients and fixed BatchNorm statistics) and only the gate is trained (learning rate 3×10^{-4}), followed by joint fine-tuning of all components with a smaller learning rate. The batch size is 32. Optuna tuned the joint learning rate, T , λ_3 , λ_4 and a (Table XIII). Each trial ran one warm-up and four joint epochs and was scored on the validation objective $L1 + (1 - \text{SSIM})$. Trial 0 used the starting values suggested in the brief ($a = 0.8$, $\lambda_3 = 0.1$, $\lambda_4 = 0.01$, $T = 1$). Trials were pruned by the median rule, or when any branch received less than 5% of the average weight on the balanced validation set (routing collapse).

Twenty-one trials were recorded: 16 completed, 4 were pruned and 1 was interrupted by a session disconnect and never finished. The brief’s starting values scored about 0.26, and the best trial reached 0.2087 (Fig. 18). The fANOVA importance assigns 0.93 to the joint learning rate and almost nothing to the other parameters. The trial logs reveal a clear trade-off that the importance plot hides. Trials with a large CE weight or a high temperature (for example trial 10: $\lambda_3 = 0.29$, $T = 2.7$)

TABLE XIII
TASK 3 OPTUNA SEARCH SPACE AND THE SELECTED CONFIGURATION (TRIAL 13).

Hyperparameter	Search range	Selected
Joint learning rate	log-uniform $[10^{-5}, 3 \times 10^{-4}]$	2.09×10^{-4}
Temperature T	log-uniform $[0.3, 3]$	0.607
CE weight λ_3	log-uniform $[0.01, 1]$	0.0115
Balance weight λ_4	log-uniform $[10^{-3}, 1]$	0.0378
Reconstruction weight a	uniform $[0.5, 0.95]$	0.703
Validation objective		0.2087

kept the gate’s argmax accuracy near 0.999 but reached only about 0.80 validation SSIM. Trials with $\lambda_3 \approx 0.01$ to 0.02 let the identity branch take more than half of the weight, lowering argmax accuracy to about 0.6, and reached 0.83. The objective therefore rewarded a gate that stops behaving like a classifier and starts acting as a mixer. Trial 15 was pruned by the collapse rule after the blur branch fell to 4.8% of the weight.

The final model was trained with three warm-up epochs and up to 35 joint epochs with patience 8. It stopped after

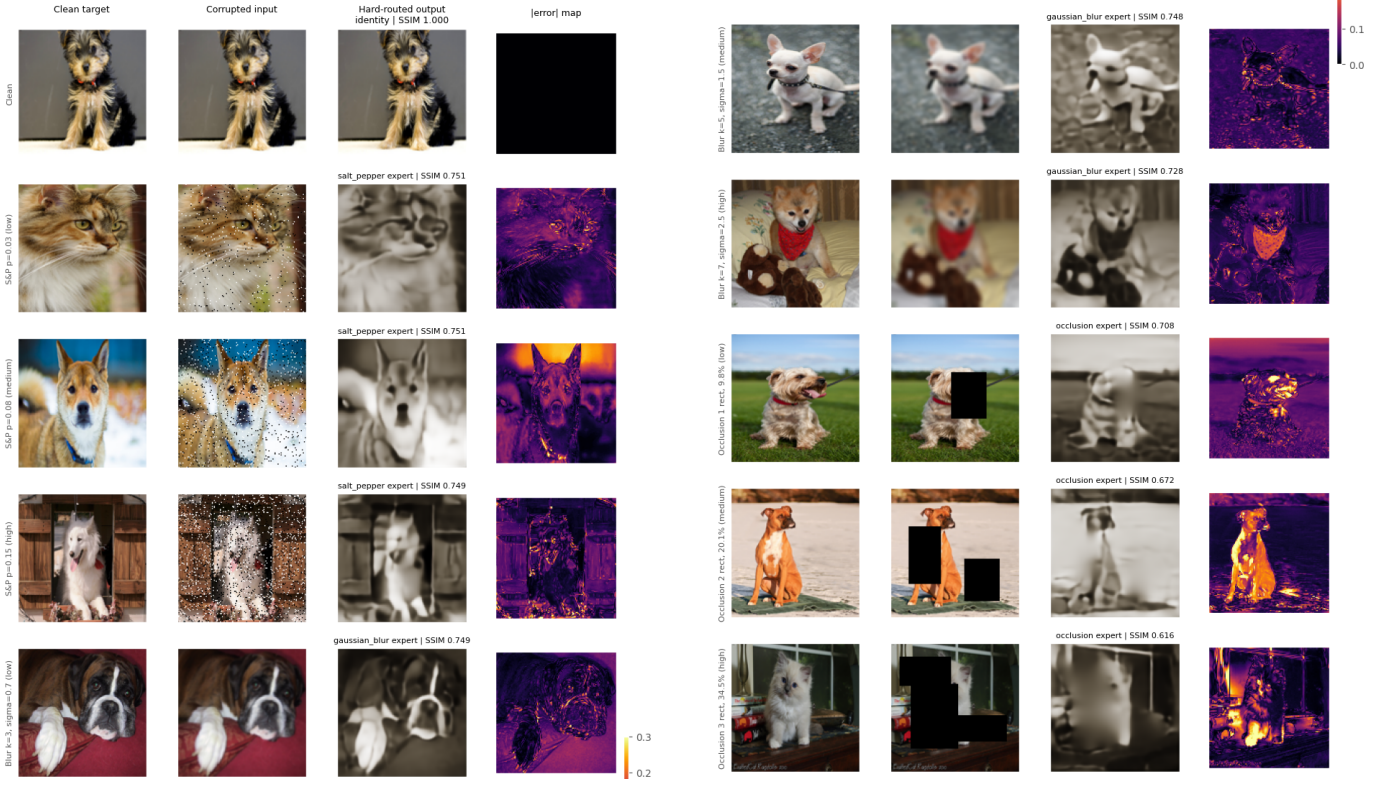


Fig. 16. Representative Task 2 results with predicted routing: clean target, model input, hard-routed output with the selected branch and SSIM, and absolute error map, for every condition and severity.

epoch 37; the best checkpoint is from epoch 29, with validation objective 0.1968 and validation SSIM 0.8394 (Fig. 19). Most of the improvement happened during warm-up, while the experts were still frozen. Validation SSIM rose from 0.783 (the hard-routing starting point) to 0.825 after three warm-up epochs, only because the gate learned to mix the identity branch with the experts. Joint fine-tuning then added another 0.015. Over the same period the share of weight on the identity branch grew from about 0.41 to 0.59 on the balanced validation set.

D. Restoration quality

On the validation manifest (736 images, 184 per condition, identical inputs for both systems), the soft mixture of experts reaches SSIM 0.839, against 0.783 for Task 2 hard routing with predicted labels and 0.783 with oracle labels: a gain of 0.056 from soft weighting alone. Since oracle routing is no better than predicted routing, the gain does not come from correcting classifier errors. It comes from blending.

E. Routing behaviour

Table XIV and Fig. 20 give the mean routing weight for every true corruption and severity on the test set, and Fig. 21 shows the distributions. The gate learned three distinct behaviours:

- **Clean images** go almost entirely to the identity branch (0.987), as in Task 2.

- **Salt-and-pepper noise** goes mainly to its expert, and more so as the noise increases (0.80, 0.95 and 0.98 for the three levels). Noise is the one corruption the expert removes far better than leaving the input alone.
- **Blur and occlusion** are blends of the identity branch and the matching expert, and the weight moves towards the expert as severity increases. Identity receives 0.88, 0.74 and 0.63 for the three blur levels, and 0.70, 0.55 and 0.41 for the three occlusion levels.

This severity-dependent blending is exactly what the Task 1 and Task 2 results suggested. Every expert reconstructs through a bottleneck that limits output SSIM to about 0.73, so for mild damage the input itself is closer to the clean image than any reconstruction. The gate discovered this without being told.

Inactive or dominant experts. Table XV summarises how much each branch is used. No branch collapsed completely, but the experts are used very unevenly. The blur expert is nearly inactive as a decision-maker: its mean weight is 0.074 and it is the strongest branch for only 0.1% of test images (its largest weight anywhere is 0.567), so blurred images are restored mostly by keeping the input. The identity branch is dominant: it receives a mean weight of 0.465 on inputs that are not clean, which the automatic check flags as dominance over unrelated inputs. The experts themselves stay specific: none receives more than 0.005 weight on corruptions other than its own. The behaviour follows from the selected loss

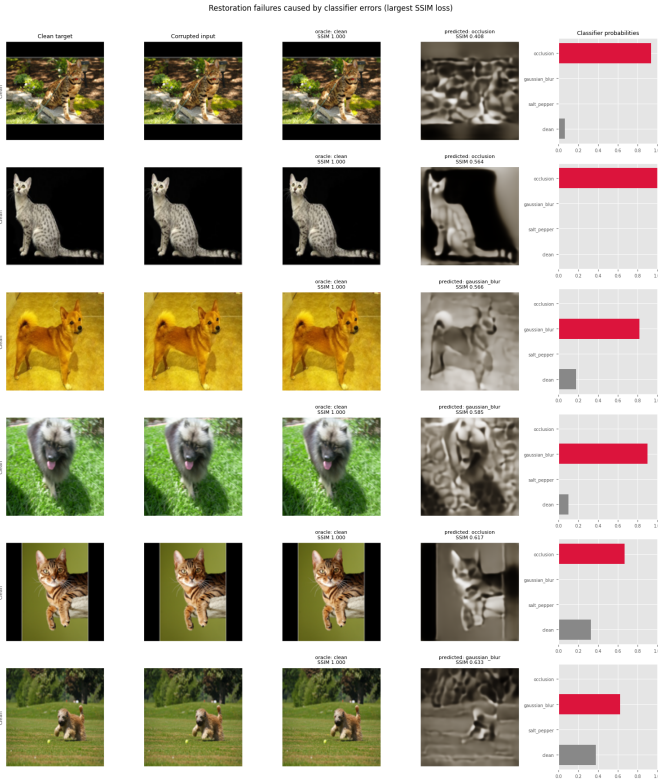


Fig. 17. Restoration failures caused by classifier errors: clean images routed to a specialist, with oracle and predicted outputs and the classifier probabilities.

TABLE XIV

MEAN ROUTING WEIGHT BY TRUE CORRUPTION AND SEVERITY (TEST SET).

True	Severity	Identity	S&P exp.	Blur exp.	Occl. exp.
Clean		0.987	0.000	0.002	0.011
Salt-and-pepper	low	0.192	0.802	0.000	0.006
	medium	0.053	0.946	0.000	0.001
	high	0.023	0.977	0.000	0.000
Gaussian blur	low	0.884	0.000	0.111	0.005
	medium	0.739	0.000	0.256	0.004
	high	0.627	0.000	0.368	0.004
Occlusion	low	0.695	0.000	0.000	0.305
	medium	0.555	0.000	0.000	0.445
	high	0.413	0.000	0.000	0.587

weights. With $\lambda_3 = 0.0115$ the label constraint is weak, and the balance term only penalises averages over a batch, so it cannot stop a branch from being used as a partial blend for most images.

Dominant and distributed examples. Fig. 22 shows the test images with the lowest and highest routing entropy. When one branch dominates, the result is clean and predictable: a clean image passes through unchanged (SSIM 0.999) and medium salt-and-pepper noise goes entirely to its expert. The other two dominant cases show the limit of optimising SSIM. A lightly blurred image is left blurred (SSIM 0.811), and a lightly occluded image keeps its black box (SSIM 0.892),

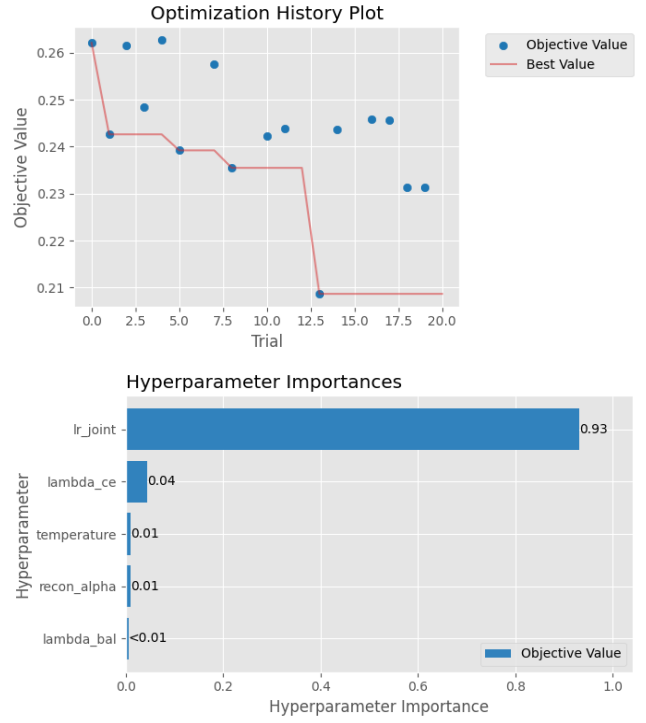


Fig. 18. Task 3 Optuna study: objective per trial (top) and fANOVA importance (bottom).

TABLE XV
BRANCH USAGE ON THE TEST SET.

Branch	Mean weight	Strongest for	Own type	Other types
Identity	0.517	59.3%	0.987	0.465
Salt-and-pepper	0.273	28.6%	0.908	0.000
Gaussian blur	0.074	0.1%	0.245	0.000
Occlusion	0.137	12.0%	0.446	0.005

because the gate scores these as better than any reconstruction even though a person would prefer the box to be filled. Distributed weights produce visible ghosting. A 0.5/0.5 blend of input and occlusion expert gives semi-transparent grey boxes, and an identity-heavy blend for light noise leaves part of the noise in the output (SSIM 0.593). The last example is the dark-background cat that Task 2 misrouted to the occlusion expert (SSIM 0.564 there). Here the gate splits its weight evenly between identity and the occlusion expert, which halves the damage (SSIM 0.618) instead of committing to the wrong branch.

Two corruptions in one image. Hard routing can only choose one expert, so 300 test images were also corrupted twice at medium severity (Table XVI). For blur plus occlusion the gate blends identity (0.53) with the occlusion expert (0.47) and beats hard routing (0.644 against 0.632). For blur plus noise it sends almost everything to the noise expert (0.994), which also removes the blur, and matches hard routing. For noise plus occlusion it mixes identity (0.55) and the noise expert (0.44); the identity share keeps part of the noise, so

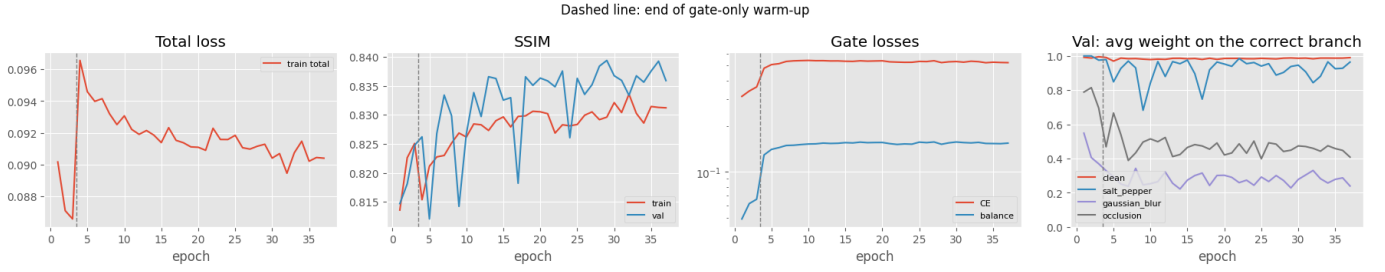


Fig. 19. Task 3 final training. The dashed line marks the end of the gate-only warm-up. Right: average validation weight on the correct branch for each true condition.

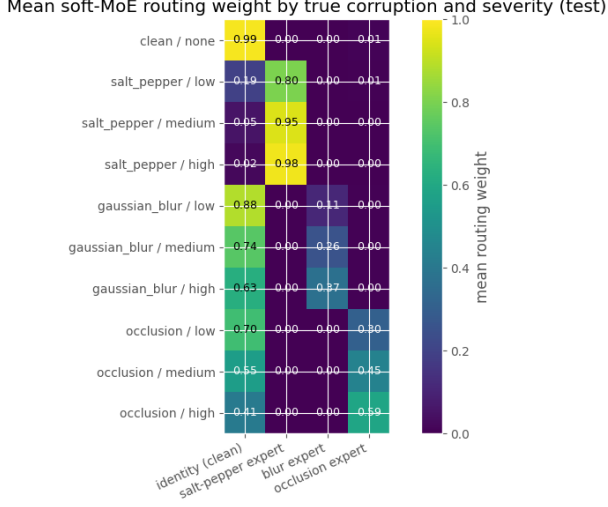


Fig. 20. Routing heatmap: mean weight of each branch for every true corruption and severity (test set).

TABLE XVI
IMAGES WITH TWO CORRUPTIONS (300 TEST IMAGES PER PAIR): SSIM AND MEAN SOFT-MO_E WEIGHTS.

Pair	Input	Hard	Soft	w_{id}	Main expert
Blur + occlusion	0.588	0.632	0.644	0.53	occl. 0.47
Blur + noise	0.241	0.697	0.691	0.01	S&P 0.99
Noise + occlusion	0.254	0.435	0.390	0.55	S&P 0.44

soft routing is worse (0.390 against 0.435). The blur expert is never chosen for these combinations, which matches its low use in Table XV.

VII. TASK 4: STYLE-CONDITIONED FACE-TO-SKETCH GENERATION

A. Architecture

Fig. 23 summarises the model. The generator $\hat{y} = G(x, s)$ is a pix2pix U-Net [11], [12] with seven downsampling and seven upsampling stages (128 to 1 and back). Encoder stages use 4×4 strided convolutions, BatchNorm and LeakyReLU (0.2) with channel widths $c, 2c, 4c, 8c, 8c, 8c, 8c$ for base width $c = 48$; BatchNorm is omitted at the first stage and at the 1×1 innermost stage. Decoder stages use transposed convolutions,

BatchNorm and ReLU, with dropout in the first three. A skip connection joins every encoder stage to the decoder stage of the same resolution, and a final transposed convolution with tanh outputs the sketch. Unlike the Task 1 autoencoder, skip connections are wanted here: the sketch must follow the facial geometry of the photograph exactly, and the skips carry that spatial detail directly to the decoder. The generator has 23,560,095 parameters.

The style condition $s \in \{1, 2, 3\}$ is a learned embedding $e(s) \in \mathbb{R}^E$ with $E = 4$. Following the style-vector expansion of FSGAN [14], $e(s)$ is tiled into an $E \times H \times W$ map and concatenated with the photograph at the generator input, and concatenated again at the 1×1 bottleneck so that the deepest features also see the style.

The discriminator is a 70×70 PatchGAN [12] that receives the photograph, a real or generated sketch, and its own tiled style embedding (concatenated as $6 + E$ input channels). Four 4×4 convolution stages ($c, 2c, 4c, 8c$; the last with stride 1) and a final 1-channel convolution produce a 14×14 map of real/fake logits for a 128×128 input, so each logit judges one local patch. The discriminator has 1,563,517 parameters. Giving the discriminator the style is necessary: without it, a sketch in the wrong style could still look real, and the generator would have no incentive to follow the condition. A projection discriminator [15] was considered as an alternative way of conditioning, but concatenation was kept because it is the method validated on FS2K by its authors [14].

B. Loss functions

The discriminator is trained with binary cross-entropy on logits, real pairs labelled 1 and generated pairs labelled 0:

$$\mathcal{L}_D = \frac{1}{2} [\text{BCE}(D(x, s, y), 1) + \text{BCE}(D(x, s, G(x, s)), 0)]. \quad (5)$$

The factor $\frac{1}{2}$ slows the discriminator relative to the generator, as in pix2pix [12]. The generator uses the non-saturating adversarial loss plus an L1 reconstruction term:

$$\mathcal{L}_G = \text{BCE}(D(x, s, G(x, s)), 1) + \lambda_{L1} \mathcal{L}_1(y, G(x, s)). \quad (6)$$

The adversarial term pushes the output towards realistic stroke texture, and the L1 term keeps it close to the paired ground truth. The commonly used value $\lambda_{L1} = 100$ [12] was the starting point; the final value was selected by Optuna. The

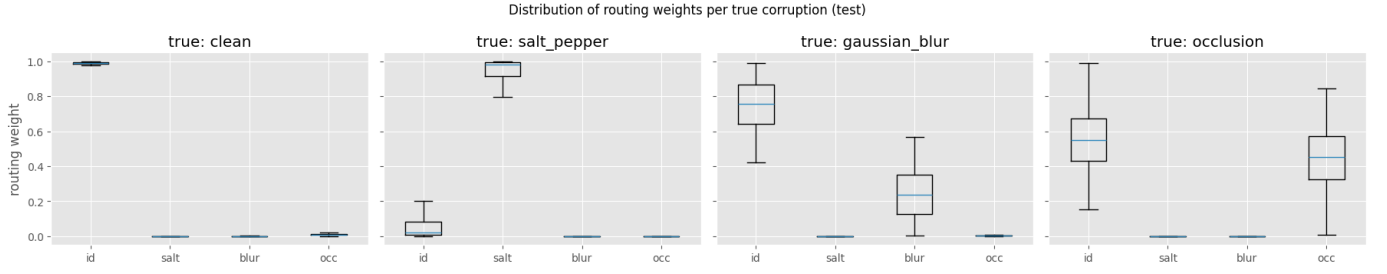


Fig. 21. Distribution of the four routing weights for each true corruption (test set).

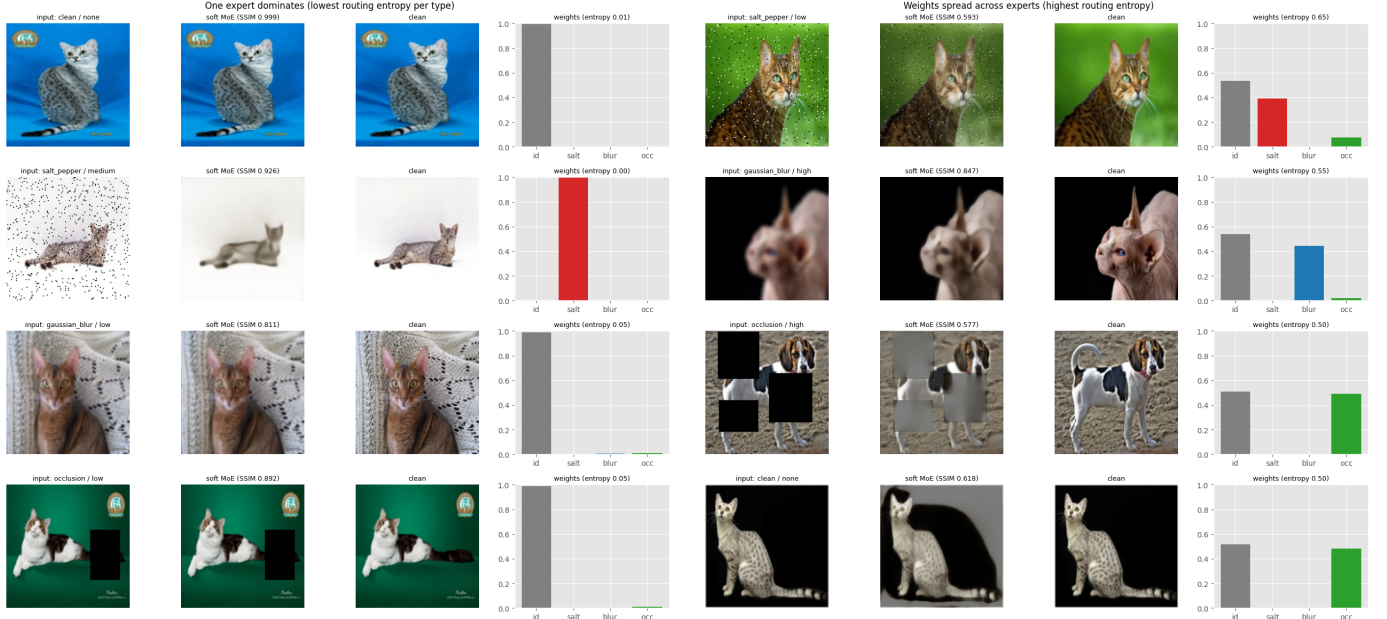


Fig. 22. Soft MoE test examples with routing weights. Left: one branch dominates (lowest routing entropy per type). Right: weights spread across branches (highest entropy).

discriminator real loss, discriminator fake loss, generator adversarial loss and generator L1 loss were logged separately at every epoch.

C. Training procedure

Weights were initialised from $\mathcal{N}(0, 0.02)$ and both networks were optimised with Adam [25] using $\beta_1 = 0.5$, $\beta_2 = 0.999$, the setting recommended for stable GAN training [26]. Each iteration performs one discriminator update followed by one generator update. The final model was trained for 200 epochs: a constant learning rate for 100 epochs, then linear decay to zero, as in pix2pix. Validation metrics were computed every 5 epochs and the checkpoint with the best validation objective was kept. The same nine validation photographs (three per style) were generated every 10 epochs and logged to Weights & Biases.

At inference the generator runs in evaluation mode, so dropout is disabled. Pix2pix keeps dropout active at test time as a source of variation [12], but a deterministic output was preferred here so that the application gives the same sketch

for the same photograph and the ONNX model can be verified exactly against PyTorch.

D. Hyperparameter optimisation

Optuna [21] ran 16 trials of 20 epochs each with the TPE sampler [22] (seed 42) and a median pruner (4 start-up trials, no pruning before epoch 8), with validation every 4 epochs. Trial 0 was fixed to the standard pix2pix settings as a reference. The selection objective was

$$J = \text{LPIPS}_{\text{val}} + (1 - \text{SSIM}_{\text{val}}), \quad (7)$$

which is fixed across trials and independent of λ_{L1} . LPIPS [17] rewards perceptual similarity of stroke texture and SSIM rewards facial structure. A pixel objective such as validation L1 was rejected because it would favour very large λ_{L1} and therefore blurry sketches. Table XVII lists the search space.

Eight trials completed and eight were pruned. The best trial (trial 12) reached $J = 0.8122$, against 0.8308 for the pix2pix reference. The four best completed trials (9, 10, 12 and 13) all used $c = 48$, batch size 8 and $E = 4$, and the two best both

U-Net generator $G(x, s)$: encoder (blue), decoder (green); dashed arcs are skip connections; numbers are spatial sizes.

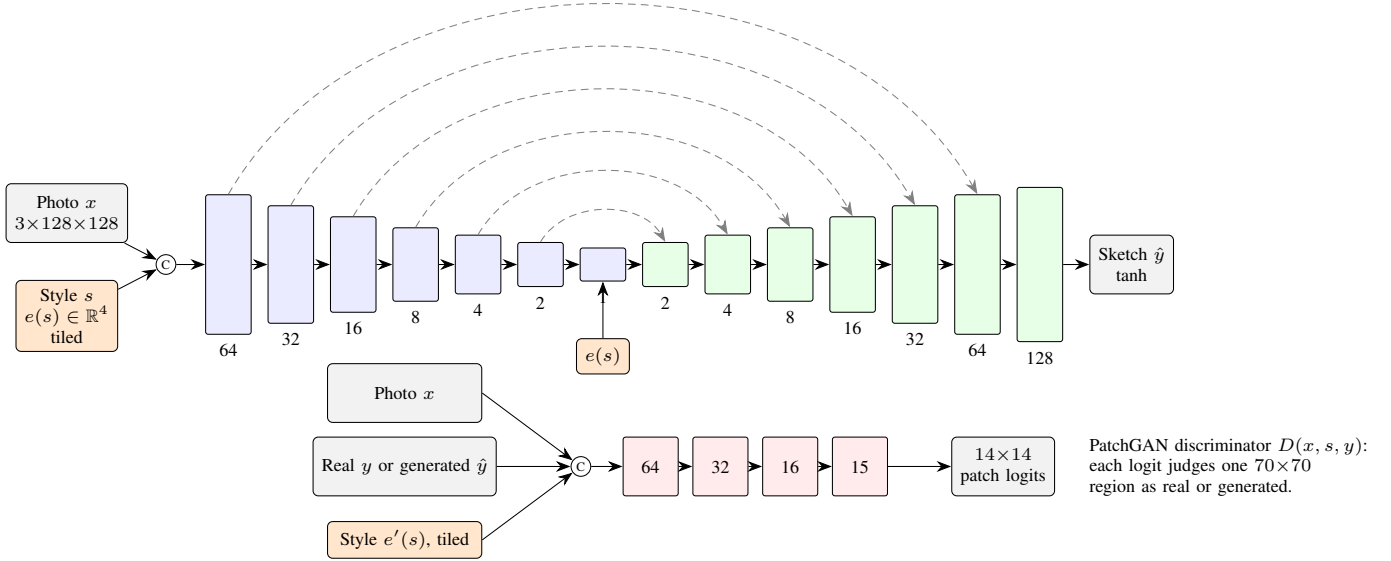


Fig. 23. Task 4 architecture. The learned style embedding is tiled and concatenated (C) at the generator input and bottleneck, and the discriminator has its own style embedding at its input.

TABLE XVII
TASK 4 OPTUNA SEARCH SPACE, PIX2PIX REFERENCE (TRIAL 0) AND
BEST TRIAL (TRIAL 12).

Hyperparameter	Range	Trial 0	Trial 12
Generator LR	$\log [5e-5, 5e-4]$	$2.0e-4$	$4.78e-4$
Discriminator LR	$\log [2e-5, 5e-4]$	$2.0e-4$	$3.79e-4$
Batch size	$\{4, 8, 16\}$	8	8
Base channels c	$\{32, 48, 64\}$	64	48
Dropout	$[0, 0.5]$	0.50	0.43
Style embedding E	$\{4, 8, 16, 32\}$	8	4
λ_{L1}	$\log [10, 200]$	100	23.3
Objective J (20 epochs)		0.8308	0.8122

used $\lambda_{L1} \approx 23$. Optuna therefore preferred a much smaller L1 weight than the usual 100, which gives the adversarial term more influence over stroke texture. The two worst completed trials (2 and 3, $J = 1.016$ and 0.908) had the smallest discriminator learning rates (3.6×10^{-5} and 2.3×10^{-5}). A slowly learning discriminator gives the generator a weak adversarial signal, so the generator falls back on the L1 term and produces flatter sketches.

The fANOVA importances [23] (Fig. 24) assign 0.28 to the generator learning rate, 0.25 to λ_{L1} , 0.25 to dropout and 0.22 to the discriminator learning rate, and less than 0.01 to the categorical parameters. These estimates come from only eight completed trials, and the categorical parameters hardly varied among them, so their low importance means that the search did not resolve their effect rather than that they have none. The study was limited to 16 trials by GAN training cost; a longer search would make these estimates more reliable.

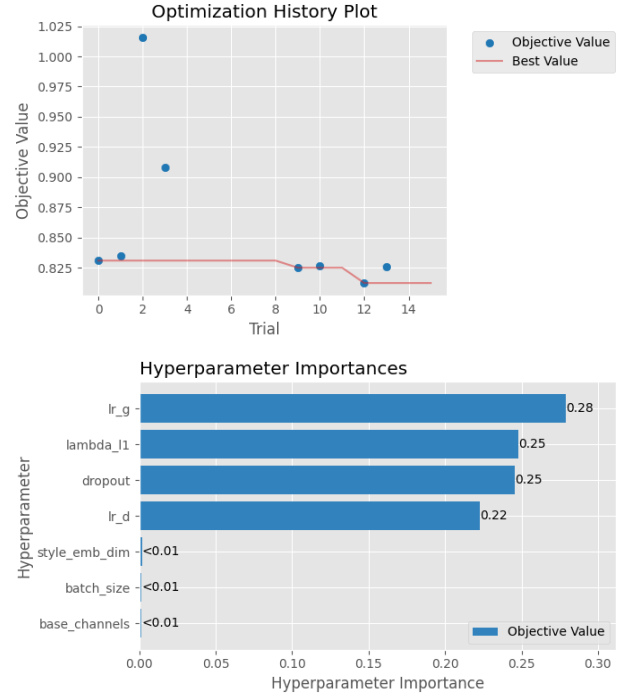


Fig. 24. Task 4 Optuna results: objective per trial (top; pruned trials have no value) and fANOVA hyperparameter importance over completed trials (bottom).

E. Training behaviour

Fig. 25 shows the separately logged losses for the final run. The generator L1 loss fell quickly from 0.30 to about 0.20 within 50 epochs and then decreased slowly to 0.191.

TABLE XVIII

TASK 4 RESULTS ON THE OFFICIAL FS2K TEST SET (TRUE STYLE).
LOWER IS BETTER FOR L1, LPIPS, FID AND KID.

Subset	n	L1	PSNR	SSIM	LPIPS	FID	KID
Overall	1,046	0.107	15.54	0.463	0.211	37.6	0.030
Style 1	619	0.082	17.24	0.504	0.214	42.3	0.033
Style 2	381	0.152	12.46	0.380	0.211	46.8	0.040
Style 3	46	0.069	18.10	0.591	0.164	80.3	0.046

Validation LPIPS dropped from 0.414 at epoch 5 to about 0.21 by epoch 50 and then levelled off near 0.20, while validation SSIM settled between 0.45 and 0.48. The validation objective was 0.7230 at epoch 100 and 0.7227 at epoch 200, so the second half of the schedule, with its decaying learning rate, refined the model only marginally. The selected checkpoint is from epoch 200.

The adversarial balance changed steadily during training. Both discriminator losses fell from about 0.6 to 0.19, the generator adversarial loss rose from about 1.1 to 2.3, and the discriminator patch accuracy on both real and generated sketches rose from about 0.70 to 0.96 (Fig. 26). The discriminator therefore became increasingly confident. This did not damage the generator: validation SSIM and LPIPS did not deteriorate, because the L1 term keeps the generator anchored to the paired target. The short spikes in the discriminator loss before epoch 50 (with matching dips in the generator adversarial loss) were transient and recovered within a few epochs. A discriminator that wins this clearly does, however, supply less useful gradient late in training; one-sided label smoothing [27], spectral normalisation [28] or a faster decay of the discriminator learning rate are possible remedies.

Fig. 27 shows the generator on fixed validation photographs. After one epoch the outputs are blurred blobs with a regular grid pattern, the checkerboard artefact typical of transposed convolutions early in training [29]. By epoch 40 the outputs are recognisable sketches with the correct overall tone for each style: light outlines for Style 1 and dark, heavily shaded hair for Style 2. Later epochs mainly sharpen strokes and facial details, and the differences between epochs 120 and 200 are small, which agrees with the validation plateau.

F. Test results

Each test photograph was generated with its true style. Table XVIII reports pixel (L1, PSNR), structural (SSIM), perceptual (LPIPS) and distribution-level (FID, KID) metrics. FID and KID compare Inception features of all generated sketches with those of all real sketches [18], [20].

Style 2 has by far the lowest PSNR and SSIM, yet its LPIPS (0.211) is as good as Style 1's. Style 2 sketches consist of dense, dark hatching (Fig. 2); a hatching pattern that is perceptually correct but whose individual strokes are shifted by a few pixels is heavily penalised by pixel and structural metrics, but much less by a perceptual metric. Pixel metrics alone would therefore understate the quality of Style 2. Conversely, Style 3 has the best SSIM and LPIPS but the

TABLE XIX

STYLE CONTROL ON THE TEST SET: LPIPS TO THE REAL SKETCH WHEN EACH PHOTOGRAPH IS GENERATED IN EACH STYLE (LOWER IS BETTER), AND HOW OFTEN THE TRUE STYLE IS BEST.

True style	n	as Style 1	as Style 2	as Style 3	True style best
Style 1	619	0.214	0.258	0.235	72.9%
Style 2	381	0.294	0.211	0.242	90.8%
Style 3	46	0.203	0.192	0.164	95.7%

worst FID (80.3). FID is biased upwards for small samples [19], and only 46 Style 3 test sketches are available, so this value mostly reflects sample size; KID, which is unbiased [20], shows a much smaller gap (0.046 against 0.033 for Style 1). The KID standard deviation for Style 3 is zero only because all 46 images fit in a single KID subset. Because 59% of the test set is Style 1, the overall figures are weighted towards that style.

Does the style condition control the output? Every test photograph was also generated in all three styles and compared with its real sketch (Table XIX). If the generator ignored the condition, the three outputs would be identical and match the real sketch equally well. Instead, the true style gives the lowest LPIPS for 72.9% of Style 1, 90.8% of Style 2 and 95.7% of Style 3 photographs, and the three outputs for the same photograph differ by a mean absolute intensity of 0.095 to 0.115. The embedding therefore genuinely conditions the generator rather than acting as an interface label. Style 1 is separated least clearly: its closest competitor is Style 3 (LPIPS 0.235 against 0.214), which is consistent with both being lighter line styles, whereas Style 2 differs strongly from both.

Qualitative results. Fig. 28 shows typical results (the test images closest to the median LPIPS of each style). The generator keeps identity, head shape, hairline and accessories such as glasses, and changing the style condition visibly changes stroke weight and shading in the expected direction. The main weaknesses are fine detail: generated hatching is simpler and more uniform than the artist's (most visible in Style 2 hair), eyes and mouths are sometimes softened, and thin glasses frames can be partly lost (fifth row).

Failure cases. Fig. 29 shows the six test images with the highest LPIPS. All six are Style 1, whose sparse outlines make any extra stroke expensive, and they share a small set of causes. (i) Several faces in the frame (first two rows): the artist drew only the main subject, but the generator sketches every face and part of the background. (ii) A very dark, low-contrast photograph (third row): facial features cannot be recovered and the generator produces unstructured texture. (iii) A small subject in a wide scene with a detailed background (last row): background structures are drawn as strokes. (iv) Unusual colour casts and hairstyles (fourth and fifth rows): the hair shape is wrong and the face interior is under-drawn. In the first row the artist also omitted the head covering, so part of the error reflects artistic interpretation rather than a model mistake. FS2K faces are mostly centred and well lit, so these

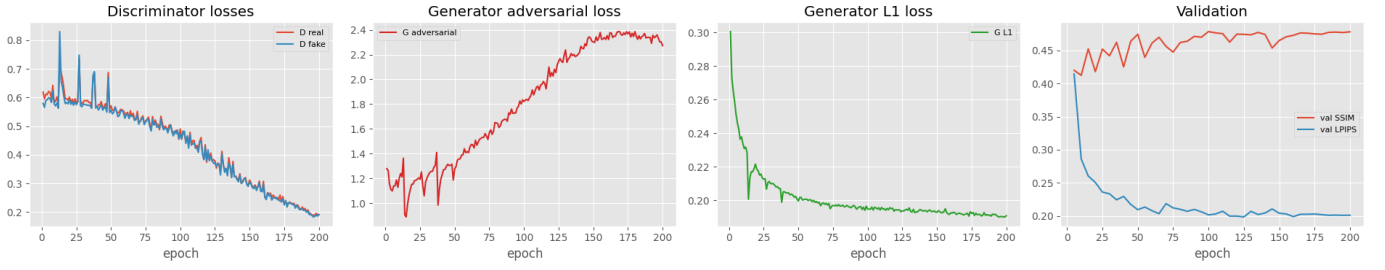


Fig. 25. Task 4 final training run: discriminator real and fake losses, generator adversarial loss, generator L1 loss, and validation SSIM and LPIPS (computed every 5 epochs).

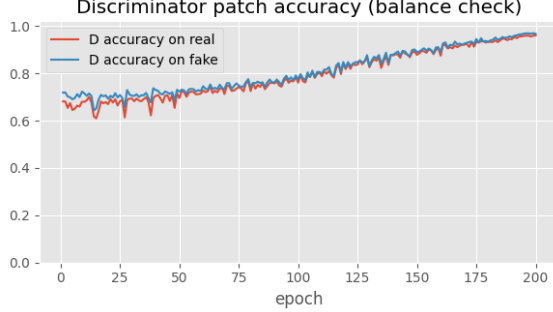


Fig. 26. Discriminator patch accuracy on real and generated sketches during the final run.

cases are out of distribution; a face detection and cropping step before inference would reduce them, and is relevant for webcam input in the application.

G. ONNX export

Only the generator is exported; the discriminator is a training component. A wrapper converts inputs from $[0, 1]$ to $[-1, 1]$ and outputs back to $[0, 1]$, so the ONNX model (opset 17, dynamic batch size) takes `photo` ($N \times 3 \times 128 \times 128$, RGB in $[0, 1]$) and `style` (N , int64 in $\{0, 1, 2\}$) and returns `sketch` in $[0, 1]$. On 60 test photographs, the maximum absolute difference between ONNX Runtime and PyTorch was 1.6×10^{-6} , 2.7×10^{-6} and 1.8×10^{-6} for the three styles, which is within floating-point tolerance. The model file is 94.3 MB, and the mean CPU latency for one image in ONNX Runtime was 25.5 ms (95th percentile 29.7 ms) on the Colab CPU.

VIII. APPLICATION DESIGN AND DEPLOYMENT

A. Interface design

The interface was first designed in Google Stitch from a written specification (Fig. 30), which produced the four workspace screens in Fig. 31. The design goals were a calm, minimal interface for an evaluator who uploads an image, runs a model and inspects the output together with system information. The palette is a lavender-tinted white page, baby blue control panels and lavender accents, with a deeper lavender for primary buttons and slate navy instead of black for text; a single rounded typeface (Nunito) is used throughout.

Every screen shares one structure: a sidebar listing the four workspaces with a status dot (ready or coming soon) and a server status box, then the workspace title, a one-sentence description, a control panel on the left and the results area on the right. The Stitch screens for Hard-Routed and Soft Mixture-of-Experts Restoration introduced the probability and routing-weight bars with the selected branch highlighted, which the implementation adopted. The implementation simplified some Stitch details: decorative labels and invented system figures (for example GPU cluster statistics) were replaced by real values reported by the backend.

B. System architecture

Fig. 32 shows the deployed system. The frontend is a React single-page application styled with Tailwind CSS, built with Vite and served by nginx. nginx also forwards every request under `/api` to the FastAPI backend, so the browser talks to a single address. The backend loads every ONNX model once at start-up into an ONNX Runtime CPU session. A task whose files are missing is reported as unavailable instead of stopping the server. For each request the backend validates the upload (format, size, readability), converts it to RGB, applies the same preprocessing as in training (bicubic resizing for Tasks 1 and 2, area resizing for Task 4), runs the model and returns the images as PNG data together with timings and the relevant model outputs. The corruption code is the unchanged Task 1 module, so corruptions applied in the application are identical to those used in training and in the test manifest. The PSNR and SSIM shown in the interface use the training definitions, and the backend’s SSIM was checked to agree with the training implementation to five decimal places.

C. Workspaces

Universal Restoration lets the user choose a clean sample image or upload one, apply a corruption at a fixed test severity (low, medium, high) or a random training-range severity, optionally with a fixed seed, or upload an already corrupted image unchanged. The result is shown in a before-and-after comparison slider or as a grid with the clean target and the error map, together with inference time, total server time, PSNR and SSIM before and after restoration, and every corruption setting used. Images are displayed pixel-sharp at their true 128×128 resolution, so the evaluator sees exactly what the model received and produced.

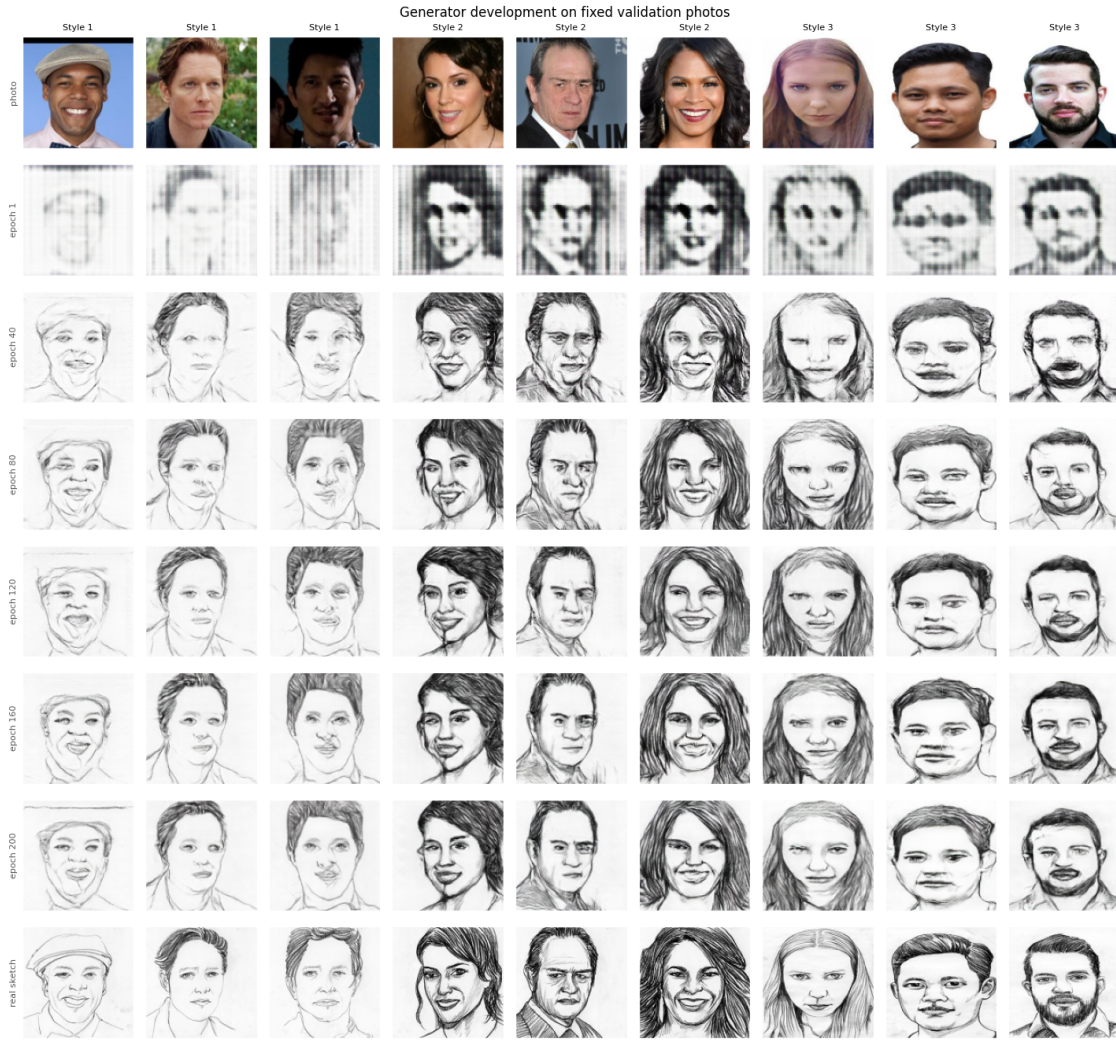


Fig. 27. Generator development on nine fixed validation photographs (three per style). Top row: input; middle rows: output at selected epochs; bottom row: the artist's sketch.

Hard-Routed Restoration uses the same inputs and adds a routing switch: classifier (predicted) routing, or oracle routing by the known corruption when the application applied it. It shows the four classifier probabilities as bars, marks the predicted class, the true corruption and the branch that actually ran, states in plain words whether the classifier was correct and whether an expert was run or the identity bypass was used, and reports classifier and specialist time separately.

Soft Mixture-of-Experts Restoration uses the same inputs and shows the four routing weights as bars. The strongest branch is highlighted, and every branch with at least 10% weight is marked as a contributor. A sentence states whether one branch dominated or the result is a blend, and the panel also reports the routing entropy, the restoration with its quality scores and the inference time of the complete ONNX pipeline (gate and all experts).

Face-to-Sketch Generator accepts an uploaded photograph or a webcam capture (centre-cropped to a square), offers Style 1, 2 or 3, and shows the photograph and the generated sketch

side by side, with an option to draw all three styles for comparison. The sketch can be downloaded at the model's 128-pixel resolution or upscaled to 512 pixels.

D. Containerised deployment

The application runs as three Docker Compose services (Fig. 32). The backend image is based on `python:3.12-slim` with pinned versions of FastAPI, ONNX Runtime, NumPy, OpenCV and Pillow, and has a health check on `/api/health`. The frontend image is built in two stages: Node 20 builds the React application, and the result is copied into an `nginx:1.27-alpine` image. A short-lived models service runs first. It checks the `models` folder and, if any file is missing, downloads the shared Google Drive folder whose address is stored in the repository's `.env` file. A failed download is reported but does not block the application, and model files are never stored in the Git repository. The whole system starts with the single command `docker compose up --build`, after which

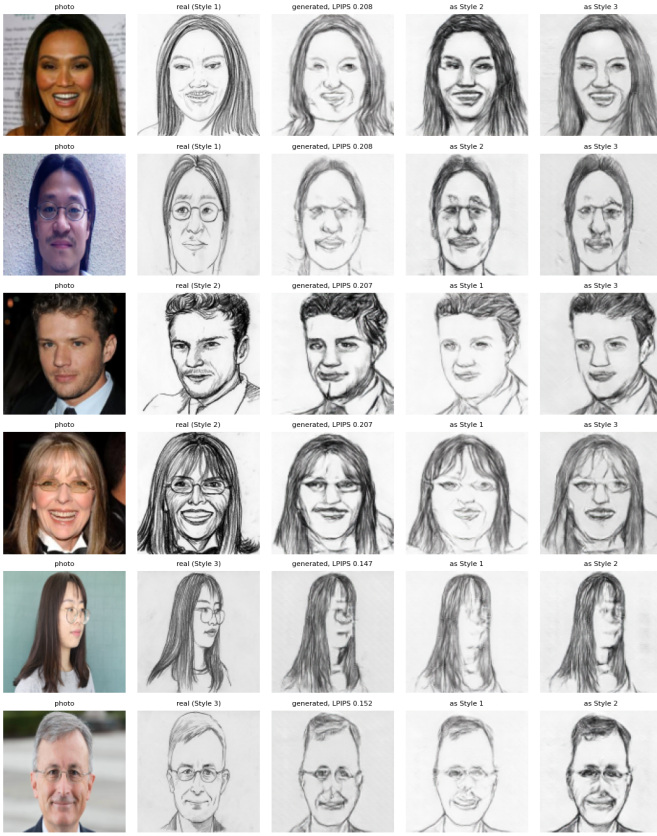


Fig. 28. Representative test results, two per style. Columns: photograph, real sketch, sketch generated with the true style, and the same photograph generated in the two other styles.

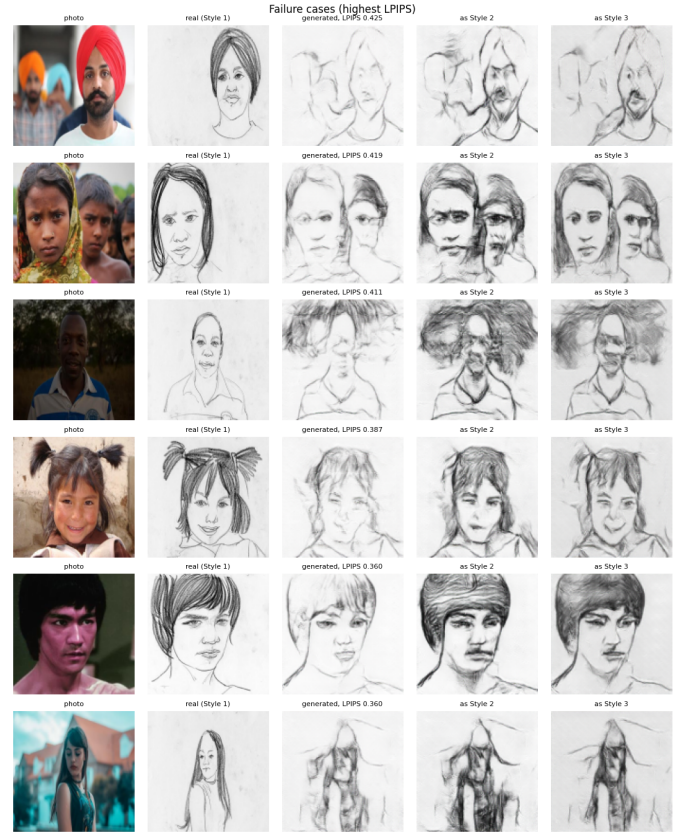


Fig. 29. Task 4 failure cases (highest test LPIPS): multiple faces, very dark lighting, unusual colour and hairstyles, and a small subject in a busy scene.

the application is available at <http://localhost:8080>. For GitHub Codespaces, where Docker’s internal network can be unavailable, an override file runs the same services on the host network and is started with `./codespaces-up.sh`.

E. Experiment tracking and reproducibility

All Optuna trials, final training runs, separately logged losses, validation sample grids, result tables, checkpoints and ONNX files were recorded in Weights & Biases (project `genai-assignment1`), grouped by task and by stage (Optuna trials, final training, test and export). Data splits, corruption manifests and Optuna studies are stored in the repository so that every experiment can be repeated.

IX. LIMITATIONS

For Tasks 1 and 2: (i) the bottleneck limits output quality to about SSIM 0.73 regardless of the input, so mild blur and low-coverage occlusion are made worse by restoration; (ii) outputs are smoothed and strongly desaturated, and highly textured scenes fail; (iii) the skip-connection ablation showed that a narrow skip would add about 0.11 SSIM, a gain deliberately not taken in order to keep a strict bottleneck; (iv) the final Task 1 model and the specialists were still improving at the end of their schedules; (v) only three corruption types, one at a time, at 128×128 were studied; and (vi) the classifier

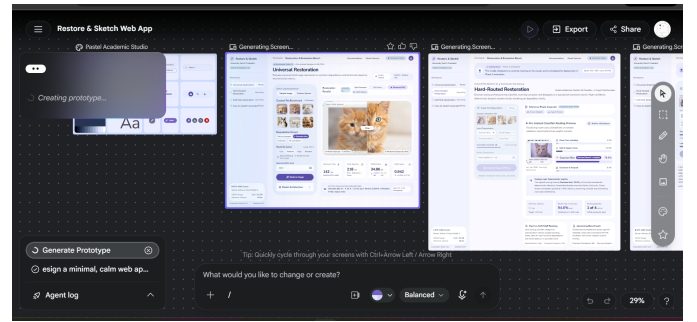


Fig. 30. The Google Stitch project: the design prompt (bottom left) and the generated screens on the canvas.

confuses clean photographs that contain large black regions or are naturally soft with occlusion and blur. For Task 3: (vii) the weak classification constraint selected by Optuna turned the gate into a mixer that is dominated by the identity branch, leaves the blur expert almost unused and keeps light occlusion boxes in place; (viii) blending produces ghosting such as semi-transparent boxes and residual noise; and (ix) every expert runs for every image, so inference costs about four times as much as hard routing. For Task 4: (i) the test set contains only 46 Style 3 pairs, so per-style statistics for Style 3, especially FID, are uncertain; (ii) the Optuna study used 16

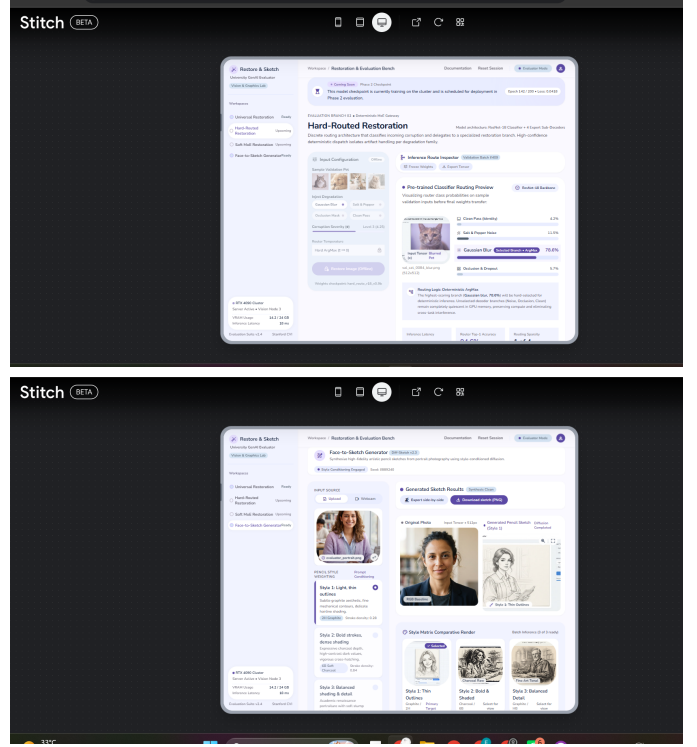
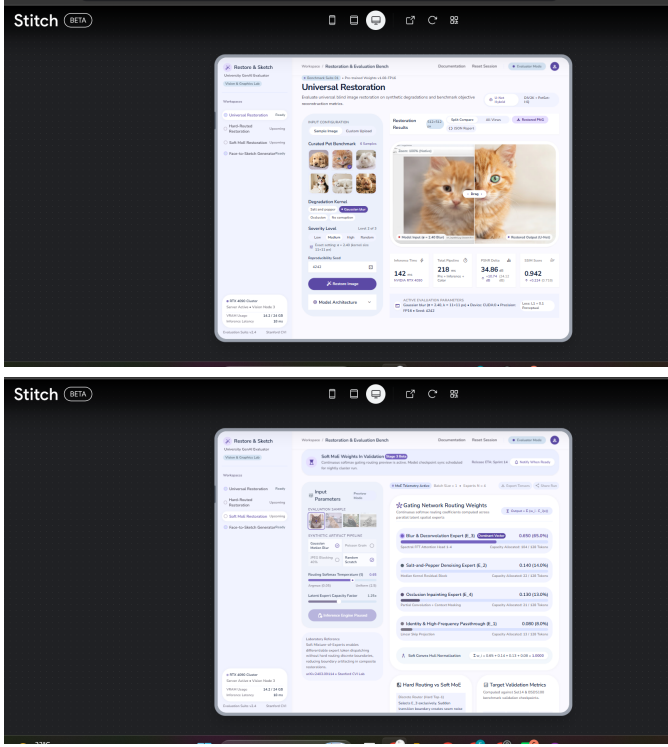


Fig. 31. Original Google Stitch designs of the four workspaces: Universal Restoration, Hard-Routed Restoration, Soft Mixture-of-Experts Restoration and Face-to-Sketch Generator.

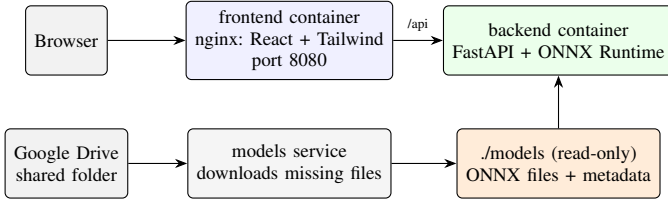


Fig. 32. Deployment: three Docker Compose services. The models service runs first and downloads any missing file; the backend serves the API; nginx serves the interface and forwards API calls.

short trials, so hyperparameter importances are only indicative; (iii) the discriminator became dominant late in training, which limits further gains from the adversarial term; (iv) generated hatching is simpler than the artists’ strokes at 128×128 ; and (v) photographs unlike FS2K (several faces, poor lighting, small faces, webcam framing) produce poor sketches without a face detection and cropping step.

X. CONCLUSION

For restoration, a single autoencoder with a 12:1 bottleneck (Task 1) removed salt-and-pepper noise well but reached the same quality ceiling, about SSIM 0.73, for every input, and so degraded clean and mildly corrupted images. Hard routing (Task 2) worked almost perfectly as a routing mechanism: the classifier reached 99.86% test accuracy, and predicted routing matched oracle routing. Its main benefit was the identity bypass, which keeps clean images intact and raised overall

TABLE XX
BACKEND ENDPOINTS (INTERACTIVE DOCUMENTATION AT [/API/DOCS](#)).

Endpoint	Purpose
GET /api/health	Server status, ONNX Runtime version, status of each model
GET /api/models	Per model: ONNX files, inputs, outputs, size, configuration, load time
GET /api/samples	Clean Oxford-IIIT Pet test images for the sample picker
POST /api/universal/restore	Task 1: image or sample, corruption, severity, seed; returns input, restoration, clean target, error map, PSNR, SSIM, settings, timings
POST /api/hard-routing/restore	Task 2: as Task 1, plus routing mode (classifier or oracle); returns the four probabilities, predicted class, branch used, correctness, and classifier and expert timings
POST /api/soft-moe/restore	Task 3: as Task 1; returns the four routing weights, strongest and contributing branches, routing entropy, restoration, scores and timings
POST /api/sketch	Task 4: photograph and style (1, 2 or 3), optionally all styles; returns the sketch at 128 and 512 pixels and timings

test SSIM from 0.704 to 0.735, while specialisation allowed a four-times smaller code at equal quality. The remaining routing errors showed that leaving lightly corrupted images untouched can beat any restoration, which motivated the weighted identity branch of Task 3. The jointly trained soft mixture of experts (Task 3) turned this observation into a learned policy: its

gate keeps clean images and lightly damaged images largely unchanged, sends noise to the noise expert, and moves weight from the identity branch to the matching expert as blur and occlusion become more severe. This raised validation SSIM from 0.783 to 0.839, at the cost of running every expert for every image, an almost unused blur expert and visible blending artefacts. Across the three strategies, a shared model is simplest but degrades easy inputs; hard routing adds reliable, cheap specialisation; and soft routing gives the best scores by deciding how much to restore, though optimising SSIM alone sometimes prefers leaving visible damage in place. For Task 4, a pix2pix-style conditional GAN with a learned style embedding in both the generator and the discriminator produced sketches that preserve identity and follow the requested style, with a test LPIPS of 0.211 and FID of 37.6. The style-control experiment showed that the embedding changes the output in the intended direction, especially for the visually distinctive Styles 2 and 3. Hyperparameter optimisation favoured a much lower L1 weight than the standard pix2pix setting, and pixel metrics proved misleading for heavily hatched styles, which supports using perceptual and distribution-level metrics for sketch synthesis.

CODE, MODELS AND VIDEO

Repository: <https://github.com/eman-huda/genai-assignment1>. The trained ONNX models are not stored in the repository; Docker Compose downloads them automatically from the shared folder configured in the repository's `.env` file, as described in its README. The demonstration video is available at <https://youtu.be/B2mOOliPXoI>.

REFERENCES

- [1] P. Vincent, H. Larochelle, Y. Bengio and P.-A. Manzagol, "Extracting and composing robust features with denoising autoencoders," in *Proc. ICML*, 2008, pp. 1096–1103.
- [2] X. Mao, C. Shen and Y.-B. Yang, "Image restoration using very deep convolutional encoder-decoder networks with symmetric skip connections," in *Proc. NeurIPS*, 2016.
- [3] K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang, "Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising," *IEEE Trans. Image Process.*, vol. 26, no. 7, pp. 3142–3155, 2017.
- [4] D. Pathak, P. Krähenbühl, J. Donahue, T. Darrell and A. A. Efros, "Context encoders: Feature learning by inpainting," in *Proc. CVPR*, 2016, pp. 2536–2544.
- [5] B. Li, X. Liu, P. Hu, Z. Wu, J. Lv and X. Peng, "All-in-one image restoration for unknown corruption," in *Proc. CVPR*, 2022, pp. 17452–17462.
- [6] R. A. Jacobs, M. I. Jordan, S. J. Nowlan and G. E. Hinton, "Adaptive mixtures of local experts," *Neural Comput.*, vol. 3, no. 1, pp. 79–87, 1991.
- [7] N. Shazeer *et al.*, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.
- [8] W. Fedus, B. Zoph and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.
- [9] I. Goodfellow *et al.*, "Generative adversarial nets," in *Proc. NeurIPS*, 2014, pp. 2672–2680.
- [10] M. Mirza and S. Osindero, "Conditional generative adversarial nets," arXiv:1411.1784, 2014.
- [11] O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. MICCAI*, 2015, pp. 234–241.
- [12] P. Isola, J.-Y. Zhu, T. Zhou and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in *Proc. CVPR*, 2017, pp. 1125–1134.
- [13] X. Wang and X. Tang, "Face photo-sketch synthesis and recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 31, no. 11, pp. 1955–1967, 2009.
- [14] D.-P. Fan, Z. Huang, P. Zheng, H. Liu, X. Qin and L. Van Gool, "Facial-sketch synthesis: A new challenge," *Mach. Intell. Res.*, vol. 19, no. 4, pp. 257–287, 2022.
- [15] T. Miyato and M. Koyama, "cGANs with projection discriminator," in *Proc. ICLR*, 2018.
- [16] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Process.*, vol. 13, no. 4, pp. 600–612, 2004.
- [17] R. Zhang, P. Isola, A. A. Efros, E. Shechtman and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. CVPR*, 2018, pp. 586–595.
- [18] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Proc. NeurIPS*, 2017.
- [19] M. J. Chong and D. Forsyth, "Effectively unbiased FID and Inception Score and where to find them," in *Proc. CVPR*, 2020, pp. 6070–6079.
- [20] M. Bifkowski, D. J. Sutherland, M. Arbel and A. Gretton, "Demystifying MMD GANs," in *Proc. ICLR*, 2018.
- [21] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD*, 2019, pp. 2623–2631.
- [22] J. Bergstra, R. Bardenet, Y. Bengio and B. Kégl, "Algorithms for hyperparameter optimization," in *Proc. NeurIPS*, 2011.
- [23] F. Hutter, H. Hoos and K. Leyton-Brown, "An efficient approach for assessing hyperparameter importance," in *Proc. ICML*, 2014, pp. 754–762.
- [24] O. M. Parkhi, A. Vedaldi, A. Zisserman and C. V. Jawahar, "Cats and dogs," in *Proc. CVPR*, 2012, pp. 3498–3505.
- [25] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.
- [26] A. Radford, L. Metz and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *Proc. ICLR*, 2016.
- [27] T. Salimans *et al.*, "Improved techniques for training GANs," in *Proc. NeurIPS*, 2016.
- [28] T. Miyato, T. Kataoka, M. Koyama and Y. Yoshida, "Spectral normalization for generative adversarial networks," in *Proc. ICLR*, 2018.
- [29] A. Odena, V. Dumoulin and C. Olah, "Deconvolution and checkerboard artifacts," *Distill*, 2016.

APPENDIX A AI USE

Generative AI tools were used as permitted by the assignment. All generated code was run, tested and checked, and all generated text was reviewed against the actual results. Table XXI lists each use.

TABLE XXI
USE OF AI TOOLS AND HOW THEIR OUTPUTS WERE VERIFIED.

Tool	Used for	Verification and corrections
Claude (Anthropic)	Task 1 notebook and shared code (corruptions, data, model, training, evaluation, ONNX export)	Corruption definitions were checked against the brief and inspected visually (Fig. 1); the achieved occlusion coverage and the balanced validation manifest were verified from printed statistics; ONNX outputs matched PyTorch within 7.2×10^{-7} .
Claude (Anthropic)	Task 2 notebook (classifier, balanced sampler, specialists, hard routing, ONNX export)	The balanced batches were verified by printing class counts per batch; ONNX outputs matched PyTorch (identical predicted classes); oracle and predicted routing were compared on the full test set.
Claude (Anthropic)	Task 3 notebook (soft MoE model, balanced sampler, joint training, routing analysis, ONNX export)	The loaded Task 2 components were checked to reproduce the Task 2 validation results before training; routing weights were checked for collapse during the search; warm-up freezing, balanced batches and weight normalisation were verified in tests.
Claude (Anthropic)	Task 4 notebook, shared modules and command-line scripts (FS2K pairing, paired augmentation, U-Net and PatchGAN with style embedding, training, evaluation, ONNX export)	Code was smoke-tested on a synthetic dataset with the FS2K folder layout before use. On the real data: the pairing report showed 0 missing pairs and 0 size mismatches; the paired augmentation figure was inspected for alignment; ONNX outputs matched PyTorch within 3×10^{-6} for all styles; training curves, generated samples and failure cases were inspected manually.
Claude (Anthropic)	Web application (FastAPI backend, React and Tailwind frontend, Docker Compose)	Every endpoint was tested with valid and invalid requests; the backend SSIM was checked against the training implementation; the application was run end to end in GitHub Codespaces.
Claude (Anthropic)	Drafting the Task 1 to 4 sections and the application section of this report from the executed notebook outputs	All numbers were checked against the notebook outputs and the interpretations against the figures. The text was reviewed by the author.
Google Stitch	Interface design from a written prompt	The generated screens were compared with the implementation; invented figures in the design were replaced by real backend values.